Gemma 4 のサービング性能

H100 NVL 1 基 / vLLM 0.32 での TTFT・Decode・MTP Drafter の実測と理論限界

2026 年 10 月

概要

Gemma 4 ファミリ（E2B, E4B, 12B, 26B-A4B MoE, 31B Dense）を NVIDIA H100 NVL 1 基上の vLLM 0.32.0 で OpenAI 互換サーバとして動かし，TTFT，TPOT，同時実行時のスループット，および公式 MTP Drafter による speculative decoding の効果を測定した．比較対象は Qwen3-30B-A3B と Qwen3-32B である．測定値はチェックポイントから計算した roofline 上の理論限界と突き合わせた．大きい dense モデルの decode は HBM 帯域の 70% に達し，MTP で最大 3.1 倍速くなる．長いコンテキストの decode は vLLM 0.32.0 では Qwen3 より遅いが，原因は FlashAttention-4 カーネルの split-KV のバグ（128 個の CTA のうち 1 個しか働かない）であり，1 行の修正で 30k トークンでの TPOT は最大 4.6 倍速くなり，Qwen3 を上回る．一方，高並列での KV キャッシュ容量とスループットでは，Hybrid Attention のアーキテクチャ上の利点を現状の vLLM が活かしきれていない．

目次

1 Gemma 4 とは 3

2 はじめに 3

3 性能分析の枠組み 4
- 3.1 Prefill と Decode . . . 4
- 3.2 roofline モデル . . . 5
- 3.3 speculative decoding のコストモデル . . . 5
- 3.4 GPU の実行モデル：SM・CTA・wave と latency . . . 6
- 3.5 計測とプロファイリングのツール . . . 7

4 実験環境とハードウェア理論値 8

5 Gemma 4 のアーキテクチャと推論上の工夫 8
- 5.1 Hybrid Attention と global 層の K=V . . . 8
- 5.2 KV 共有（E2B/E4B） . . . 9
- 5.3 Per-Layer Embeddings（PLE，E2B/E4B） . . . 11
- 5.4 MoE（26B-A4B） . . . 11
- 5.5 MTP Drafter . . . 13

6 理論モデル 14
- 6.1 Decode の読み出しバイト . . . 14
- 6.2 Decode の FLOPs とステップ時間の下限 . . . 15
- 6.3 Prefill の FLOPs . . . 15
- 6.4 KV キャッシュの格納量 . . . 16

6.5 speculative decoding のステップコスト 16
6.6 理論値の一覧 16

7 実験方法 17

8 結果 18
8.1 Prefill：Hybrid Attention は長いプロンプトで効く 19
8.2 短いコンテキストの decode：大きいモデルほど理論値に近い 19
8.3 長いコンテキストの decode：Attention カーネルの split-KV のバグ 22
8.4 split-KV を修正した FA4 での測り直し 25
8.5 同時実行：バッチによる償却と，MoE・KV 容量の限界 27
8.6 MTP Drafter による speculative decoding 30

9 外部の結果との比較と総合評価 33
9.1 外部の結果と整合しているか 33
9.2 Qwen3 と比べて良い結果か 34
9.3 コミュニティとして何をすべきか 35

10 ボトルネックとチューニングの指針 36

11 まとめ 36

付録 A 付録 37

1　Gemma 4 とは

Gemma 4 は Google DeepMind が 2026 年に公開したオープンウェイトのモデルファミリである [1, 2, 3]．端末向けの E2B / E4B から，GPU 1 基で動く 12B・26B-A4B（MoE）・31B（dense）まで 5 サイズがある（図 1）．2026 年 5 月には，speculative decoding 用の小型モデル **MTP Drafter** が全サイズ向けに追加公開された [4, 5]．

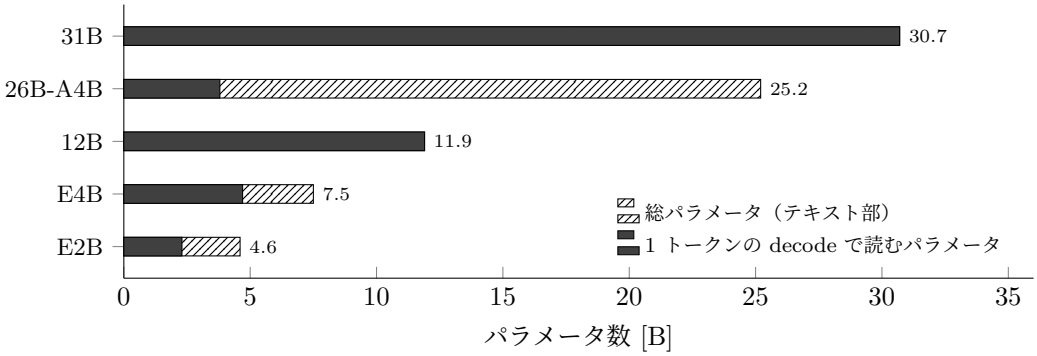


図 1: Gemma 4 ファミリ．E2B/E4B は総パラメータの約半分が層ごとの embedding テーブル（PLE）で，26B-A4B は MoE である．どちらも 1 トークンの decode で実際に読むパラメータは総数よりずっと少ない（数値は本書で測ったチェックポイントの値）．

Gemma 4 には，同じ性能をより少ない計算量とメモリで出すための工夫が多い．

- **Hybrid Attention と K=V**：大半の層を sliding window Attention にして，長いコンテキストでの KV キャッシュを減らす．global 層では value を key と同じ projection から作る（V projection の重みを持たない）．
- **KV 共有と PLE**（E2B/E4B）：後段の層が前段の KV を再利用する．巨大な embedding テーブルは lookup だけで済ませる．
- **MoE**（26B-A4B）：1 トークンで使う重みを総 25B のうち 3.8B に絞る．
- **MTP Drafter**：小型の drafter が次の数トークンを予測し，本体が 1 回の forward でまとめて verify する．

いずれも，サービスとして動かしたときの応答開始までの時間，生成速度，同時に処理できるリクエスト数に直接効くはずの設計である．ただし，その効果が実際の推論エンジン上でどれだけ表れるかは，公式資料だけでは分からない．

本書の目的

Gemma 4 の**サービング性能**を，同じ GPU・同じ推論エンジン（H100 NVL 1 基，vLLM 0.32）の上で測り，同クラスの Qwen3（30B-A3B MoE / 32B dense）と比較する．測定値はすべてチェックポイントから計算した理論限界と突き合わせ，**どこまで性能を出し切れているか**と**何が律速しているか**を数値で示す．

2　はじめに

モデルカードが報告するのは主に精度であり，推論速度は GPU と推論エンジンの組み合わせで大きく変わる．本書では Gemma 4 の設計上の工夫が，実際のサービング性能にどう表れるかを次の 4 つの問いで調べる．

Q1　Gemma 4 は GPU の性能（HBM 帯域・演算性能）をどこまで出し切れているか．

Q2　Hybrid Attention・K=V・MoE は，同クラスの Qwen3 に対して実際に速さや容量の差を生むか．
Q3　MTP Drafter はどの条件でどれだけ速くなるか．効果の差は何で決まるか．
Q4　理論値に届かない場合，何が律速しているか．

■主な結果．

1. 31B の decode は HBM 理論帯域の 70% に達し，ほぼ理論どおりに動く．小さいモデルと MoE は 28–38% にとどまる．Nsight Systems で分解すると，重みの読み出し以外にかかる時間（SM を埋めきれない小さな GEMM，normalization などの小さなカーネル群，Attention，GPU idle）はモデルの大きさによらず 1 ステップ 2.7–4.6 ms あり，読むバイトが少ない小さいモデルほどこの一定の上乗せが相対的に重くなるためである（§8.2）．
2. Hybrid Attention は長いプロンプトの prefill で効き，32k トークンの TTFT は Qwen3 より 16–33% 短い．sliding window で Attention の FLOPs が減るためで，その差は Attention が FLOPs の大きな割合を占める 4k トークン以上で現れ，window size（1024）以下では差がない（§8.1）．一方，vLLM 0.32.0 では長いコンテキストの decode が，KV の読み出し量が Qwen3 の 3–4 割しかないのに Qwen3 より遅い．プロファイルでは，global 層の Attention カーネルが理論どおりのバイト数しか読んでいないのに HBM 帯域の 1% 未満しか使っていない．原因は FA4 カーネルの split-KV に分割数が渡っていないバグで，128 個の CTA のうち 1 個だけがすべての KV を読んでいた（§8.3）．1 行の修正で 30k での TPOT は E2B で 17.0 ms から 3.7 ms，31B で 40.7 ms から 23.3 ms になり，31B は Qwen3-32B（24.9 ms）より速くなる（§8.4）．この修正は upstream にすでに入っており，v0.32.1rc0 以降で使える．
3. MTP Drafter は単一ストリームの decode を 1.35–3.1 倍速くする．効果は 31B で最大，26B-A4B で最小であり，この差は帯域のコストモデルで説明できる．
4. 短いリクエストを高並列で処理すると，31B は 64 並列で Qwen3-32B の 0.6 倍のスループットしか出ない．KV キャッシュが足りずリクエストが待たされるが，FP8 KV キャッシュで KV 容量を 1.9 倍にしても待ち時間が減るだけでスループットは変わらない（§8.5）．

■本書の構成． §3 で分析に使う枠組み（roofline モデルと speculative decoding のコストモデル）と計測ツールを導入し，§4 で実験環境のハードウェア性能を示す．§5 で Gemma 4 のアーキテクチャを推論性能の観点から整理し，§6 でそれを理論限界の式に落とす．§7 で実験方法を述べ，§8 で結果を示す．§9 では外部の結果と比べたうえで，Qwen3 に対して良い結果だったのか，コミュニティとして何をすべきかを論じ，§10 でボトルネックと対策をまとめる．

3　性能分析の枠組み

3.1　Prefill と Decode

LLM の推論 1 リクエストは，プロンプトを一括処理する **prefill** と，1 トークンずつ生成する **decode** に分かれる（図 2）[8]．本書では次の 3 つの指標を使う．

- **TTFT**（Time To First Token）：リクエストを送ってから最初のトークンが返るまでの時間．主に prefill で決まる．
- **TPOT**（Time Per Output Token）：2 トークン目以降の 1 トークンあたりの生成時間．decode 1 ステップの時間に等しい．
- **スループット**：全リクエスト合計の出力トークン数 / 秒．同時実行数を増やすと上がる．

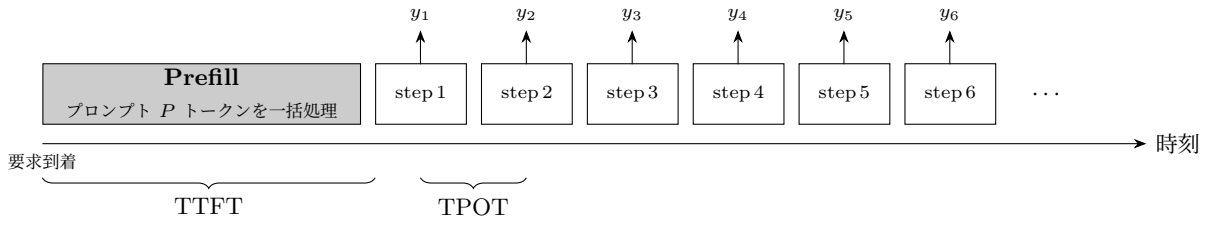


図 2: 1 リクエストの処理の流れ．prefill と decode では律速要因が異なる．

3.2 roofline モデル

ある処理が F [FLOP] の演算と Q [byte] のメモリ転送を必要とするとき，演算ピーク π [FLOP/s]，メモリ帯域 β [byte/s] の GPU での実行時間は

$$t \;\geq\; \max\!\left(\frac{F}{\pi},\; \frac{Q}{\beta}\right) \tag{1}$$

で下から抑えられる．演算と転送が完全に重なった理想的な状態でも，どちらか遅い方より速くはならないからである．両辺を F で割ると，到達できる演算性能は $F/t \leq \min(\pi,\, I\beta)$ となる．ここで $I = F/Q$ を **arithmetic intensity**（1 byte 転送あたりの演算量）と呼ぶ．これを図にしたものが roofline（図 3）で，折れ曲がる点

$$I^* = \pi/\beta \tag{2}$$

を **ridge point** と呼ぶ．$I < I^*$ ならメモリ帯域律速，$I > I^*$ なら計算律速である．式 (1) はキャッシュ，カーネル起動，CPU 側の処理を無視した下限なので，**実測値と下限の差**が「その他の要因」による損失，つまり改善の余地を表す．

■LLM への適用． BF16 の重みは 1 パラメータ 2 byte で，1 トークンの計算では各パラメータを 1 回の積和（2 FLOP）に使う．decode で B 本の系列を 1 ステップ進めるとき，重みは B 本で共有して 1 回だけ読めばよいので，$F \approx 2NB$，$Q \approx 2N$（N はパラメータ数）となり，$I \approx B$ である．同様に，P トークンの prefill では $I \approx P$ になる．この関係は LLM 推論の性能分析でよく使われるもので，文献 [8] は「バッチサイズ＝合計トークン数と考えるとシンプル」と述べ，文献 [7] は A100 では同じ議論から「バッチサイズ 208 未満の decode はメモリ帯域律速」と見積もっている．ここでの $I \approx B$ は重みの読み出しだけを数えた近似であり，KV キャッシュは系列ごとに別に読むので，コンテキストが長いと実際の I はこれより小さい．したがって decode は同時実行数 B が I^* を超えるまでメモリ帯域律速で，prefill は数百トークン以上で計算律速になる．

3.3 speculative decoding のコストモデル

speculative decoding では，軽い drafter が γ トークンを順に予測し，本体（target）が $\gamma+1$ 位置を 1 回の forward でまとめて verify する（図 4）．先頭から一致したトークンと，target 自身が出す 1 トークンが確定する．1 回の verify で確定するトークン数の平均を **mean acceptance length** AL（$1 \leq \mathrm{AL} \leq \gamma+1$）とする．greedy 生成なら出力は通常の decode と完全に一致する．

1 回の speculative step（drafter γ 回＋ verify 1 回）の時間が通常の decode 1 ステップの c 倍だとすると，c ステップ分の時間で AL トークンが確定するので，

$$\text{TPOT 高速化} = \frac{\mathrm{TPOT}_{\mathrm{base}}}{\mathrm{TPOT}_{\mathrm{spec}}} = \frac{\mathrm{AL}}{c} \tag{3}$$

となる．AL は drafter の予測精度，つまり生成内容で決まる．c は drafter と verify の**コスト**で，モデル構造と

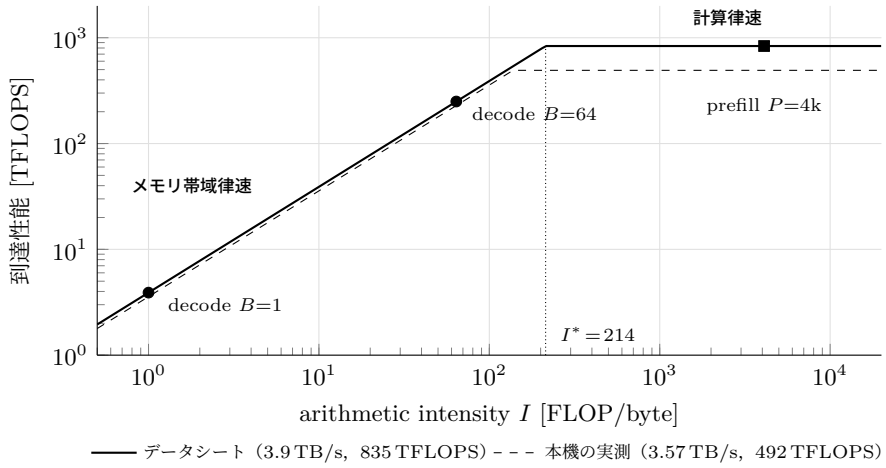


図 3: 本機（H100 NVL）の roofline と典型的な動作点．decode の arithmetic intensity はバッチサイズ B，prefill はプロンプト長 P にほぼ等しい．

ハードウェアで決まる．両者を分けて測れば，「acceptance rate が低いから遅い」のか「verify が高くつくから遅い」のかを区別できる．c の理論値は §6.5 で導く．

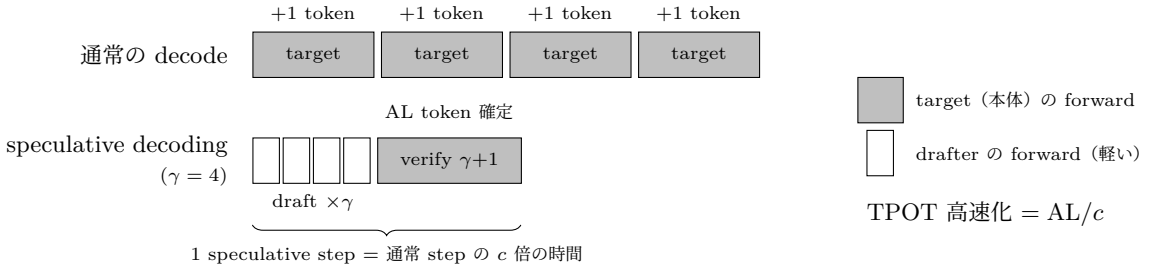


図 4: 通常の decode と speculative decoding．speculative step 1 回で平均 AL トークンが確定し，時間は通常の c 倍かかる．

3.4 GPU の実行モデル：SM・CTA・wave と latency

ルーフラインは「十分な並列度がある」ことを前提にした上限である．実測が上限から遠いとき，その原因を調べるには GPU がカーネルをどう実行するかを知っておく必要がある [17, 18].

■SM と CTA． H100 NVL は 132 個の **SM**（Streaming Multiprocessor）を持つ [16]．カーネルは **CTA**（Cooperative Thread Array，CUDA では thread block）の集まり（grid）として起動され，1 つの CTA は 1 つの SM 上で実行される．CTA は 32 thread ずつの **warp** に分かれ，SM は最大 64 warp（2048 thread）を同時に抱えて，待ちの発生した warp から別の warp に切り替えながら実行する．同時に実行される CTA の一群を **wave** と呼ぶ．NVIDIA は，CTA の数を SM の数の数倍以上にして複数の wave で実行するのが効率的だとしている [17]．CTA が SM より少ないと，仕事のない SM が生じ，その SM の演算器もメモリ帯域も使われない（図 5(a)）．

■latency hiding と Little の法則． HBM からのロードには数百 ns かかる．この待ち時間（latency）の間も帯域を埋めるには，常に大量のロードを発行中にしておく必要がある．Little の法則によれば，到達できる帯域は「発行中のバイト数 / latency」で決まる [19]．仮に latency を 500 ns とすると，3.9 TB/s を出すには約 2 MB，SM 1 個あたり十数 KB のロードを常に発行中にしておかなければならない．SM 上の warp が少ない（occupancy が低い）か，1 warp が逐次にしかロードを出さないと，この条件を満たせない（図 5(b)）．このとき処理はメモリ律

速でも計算律速でもなく，**latency 律速**になる．NVIDIA のガイドも，並列度が足りない処理はプロセッサが使い切られず latency で性能が決まるとしている [17].

■カーネル単位の固定費． カーネル 1 個の実行には，中身の計算とは別に，起動・終了と SM への CTA の割り当てにかかる固定的な時間（数 µs）がある．CUDA Graph[18] はカーネル列をまとめて登録し，CPU からの起動コストを省くが，GPU 上でのカーネルごとの固定費とカーネル間の隙間は残る．1 ステップに数百個の小さなカーネルが並ぶと，この固定費が積み上がる．

■本書での使い方． Nsight Systems からはカーネルごとの時間と grid の大きさ（CTA 数）が，Nsight Compute からは DRAM・SM の使用率とアクティブ warp の割合（achieved occupancy）が得られる．§8.2 と §8.3 では，これらを使って「CTA が SM を埋めているか」「latency を隠せるだけの warp があるか」を確かめる．

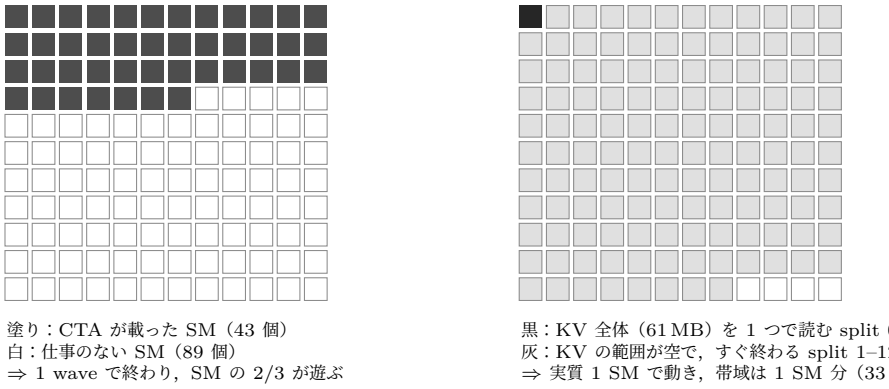


図 5: 本書で観測した 2 つの並列度不足．(a) E2B の decode の GEMM は 43 CTA しかなく，132 SM のうち 89 個に仕事がない．(b) vLLM 0.32.0 の Gemma 4 の global 層の Attention カーネルは 128 個の split-KV の CTA を起動するが，バグのため split 0 の 1 個だけが KV 全体を読み，残りの 127 個は空の範囲を受け取ってすぐに終わる．

3.5 計測とプロファイリングのツール

分析は「理論限界を計算する → 実測する → 比をとる → 限界から遠ければプロファイラで原因を特定する」の順で進める．使った道具を表 1 にまとめる．

表 1: 本書で使う計測・分析ツール．

ツール	何が分かるか	本書での用途
ストリーミングクライアント	ユーザから見た TTFT・TPOT・スループット	全実験の主指標
vLLM の `/metrics`	speculative decoding の draft 数・accepted token 数	AL と位置ごとの acceptance rate
マイクロベンチマーク	本機で実際に出る GEMM 性能と HBM 帯域	roofline の天井を実測値で引き直す
Nsight Systems	タイムライン，カーネル別の時間，GPU の idle 時間	decode 1 ステップの時間内訳
Nsight Compute	カーネル単位のハードウェアカウンタ（DRAM・SM 使用率，アクティブ warp）	律速要因（帯域・演算・latency）の判定

4　実験環境とハードウェア理論値

表 2: 実験環境．GPU の理論値は NVIDIA H100 NVL データシート [11]（1 GPU あたり）．Tensor Core 性能は dense 値（データシートの “with sparsity” 値の 1/2）．

項目	値
GPU	NVIDIA H100 NVL（PCIe Gen5，Hopper GH100，132 SM）
GPU メモリ	94 GB HBM3（95,830 MiB 認識）
HBM 帯域（理論）β	3.9 TB/s
BF16 Tensor Core（dense）π	835 TFLOPS（最大ブースト 1,785 MHz 換算の上限は 965 TFLOPS）
FP8 Tensor Core（dense）	1,671 TFLOPS
L2 キャッシュ / TDP	60 MiB / 400 W
実測（マイクロベンチ）	BF16 GEMM 最大 492–514 TFLOPS，ストリーム読み出し 3.57–3.58 TB/s，コピー 3.57–3.58 TB/s（読み + 書き，4 回の範囲）
CPU / RAM	AMD EPYC 9V84（40 vCPU）/ 314 GiB
ソフトウェア	Driver 595.91.07，CUDA 13.2，PyTorch 2.13.0+cu130，vLLM 0.32.0，Transformers 5.19

§3.2 の ridge point は，データシート値で $I^* = 835/3.9 \approx 214$ FLOP/byte，本機の実測値で $(492\text{–}514)/3.57 \approx 138\text{–}144$ FLOP/byte である．decode の arithmetic intensity は $I \approx B$ なので，dense モデルの decode は同時実行数が 140–210 程度になるまで帯域律速である．なお H100 NVL は TDP が 400 W と低く，大きな GEMM では電力制限でクロックが下がるため，実測の GEMM 性能はデータシートの 59–62% にとどまる．同じマイクロベンチを日を変えて 4 回測ると，HBM 帯域は 3.57–3.58 TB/s で安定していたが，GEMM ピークは 492–514 TFLOPS と 4% ほど変わった．電力制限によるクロックの揺れと考えられる．

5　Gemma 4 のアーキテクチャと推論上の工夫

公式テクニカルレポート [1]，モデルカード [2]，Hugging Face ブログ [3]，MTP の告知 [4, 5] から推論性能に関係する事項を抜き出し，数値は各モデルの `config.json` で確認した（表 3，図 6）．

表 3: Gemma 4 と比較モデルの構成（`config.json` より）．Q/KV ヘッドと head dim は local 層の値で，括弧内は global 層の値．

モデル	層	hidden	Q / KV heads	head dim	window	global 層	特徴
E2B	35	1536	8 / 1 (1)	256 (512)	512	7	PLE，KV 共有 20/35 層
E4B	42	2560	8 / 2 (2)	256 (512)	512	7	PLE，KV 共有 18/42 層
12B	48	3840	16 / 8 (1)	256 (512)	1024	8	K=V（global），encoder-free
26B-A4B	30	2816	16 / 8 (2)	256 (512)	1024	5	MoE 128 expert top-8 + dense MLP，K=V
31B	60	5376	32 / 16 (4)	256 (512)	1024	10	K=V（global）
Qwen3-30B-A3B	48	2048	32 / 4	128	–	48	MoE 128 expert top-8
Qwen3-32B	64	5120	64 / 8	128	–	64	dense

5.1　Hybrid Attention と global 層の K=V

local 層は直近 w トークン（512 または 1024）だけに attend するので，KV キャッシュは window size で頭打ちになる（図 7）．コンテキスト長に比例して KV が増えるのは global 層だけで，31B では 60 層中 10 層である．global 層は KV ヘッドを 1–4 個に絞って head dim を 512 に広げ，RoPE は head dim の 1/4 だけに掛け

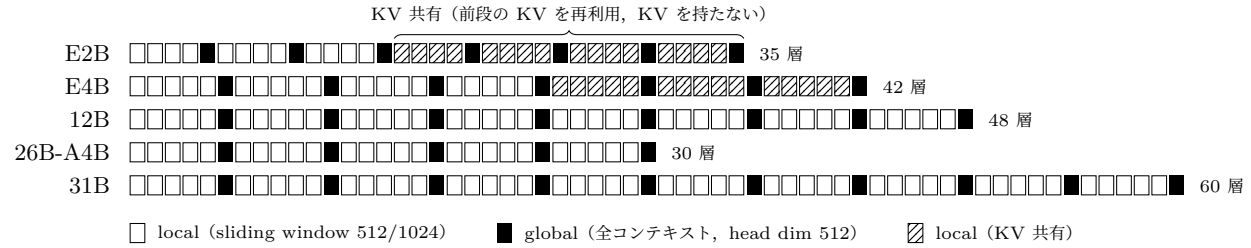


図 6: 各モデルの層構成．E2B は 4:1，ほかは 5:1 で local 層と global 層が並ぶ．E2B/E4B の後段（括弧の範囲）は KV 共有区間で，この区間の global 層も前段の global 層の KV を再利用する（詳細は図 8）．Qwen3 は全層が global である．

る（p-RoPE，$p = 0.25$）．12B/26B/31B の global 層はさらに K=V で，V projection の重みを持たず，K と同じ projection の出力から V を作る．

ただし **K=V でも，現行の実装では KV キャッシュは減らない**．Transformers と vLLM の実装を確認すると，K は projection の出力に normalization（重みあり）と RoPE を掛けたもの，V は同じ出力に重みなしの normalization だけを掛けたもので，両者は別のテンソルとしてキャッシュに格納される（vLLM は K projection の重みを V の枠にも読み込んで同じ計算をする）．K=V が減らすのは V projection の重みと計算であり，KV キャッシュの量ではない．RoPE を掛ける前の K を 1 つだけ保存し，読み出し時に V と K を作り分ければ global 層の KV を半分にできる余地はあるが，本書の理論値（§6）は実装どおり K と V の両方を数える．vLLM での実装をコード 1 に示す．window size は層ごとに Attention モジュールへ渡され，global 層では `None`（窓なし）になる．

コード 1: vLLM の Gemma 4 Attention 層（抜粋）．`# <-` 以降は筆者の注記（以下同様）．

```
# vllm/model_executor/models/gemma4.py  class Gemma4Attention (excerpt)
sliding_window = config.sliding_window if self.is_sliding else None
self.attn = Attention(..., per_layer_sliding_window=sliding_window, ...)  # <- local: window, global: None

def forward(self, positions, hidden_states, **kwargs):
    ...
    # For k_eq_v, K weights are loaded into both K and V slots of
    # qkv_proj, so V == K automatically.
    qkv, _ = self.qkv_proj(hidden_states)
    q, k, v = qkv.split([self.q_size, self.kv_size, self.kv_size], dim=-1)
    ...
    k = self.k_norm(k)                          # <- K: RMSNorm with weight
    q, k = self.rotary_emb(positions, q, k)     # <- K: RoPE
    v = self.v_norm(v)                          # <- V: RMSNorm without weight, no RoPE
    attn_output = self.attn(q, k, v)            # <- K and V are cached separately
```

推論への効果は 2 つある．1 つは長いコンテキストでの KV の格納量と読み出し量が減ること，もう 1 つは prefill の Attention 計算が local 層ではプロンプト長に比例するだけで済むことである（§6.3）．ただし window size 以下の短いコンテキストでは，ヘッド数の多い local 層の KV が効いてくる．1 トークン・1 層あたりの KV は 31B の local 層が $16 \times 256 \times 2$ 要素で，Qwen3-32B（$8 \times 128 \times 2$）の 4 倍ある．

5.2 KV 共有（E2B/E4B）

通常の Transformer では，各層が自分の K/V projection で KV を作り，KV キャッシュに保存する．E2B/E4B では後段の層（E2B は 35 層中 20 層，E4B は 42 層中 18 層）がこれを省き，KV を作る区間の**最後の同じ種類の層**の KV をそのまま使う（図 8）．E2B では，共有区間の local 層 16 個はすべて第 14 層の KV を，global 層 4 個は第 15 層の KV を参照する．共有層は K/V projection の重みを持たず，query だけを計算する（コード 2）．なお E2B では，その代わりに共有層の MLP の中間次元が 2 倍（6144 → 12288）になっている（`use_double_wide_mlp`）．

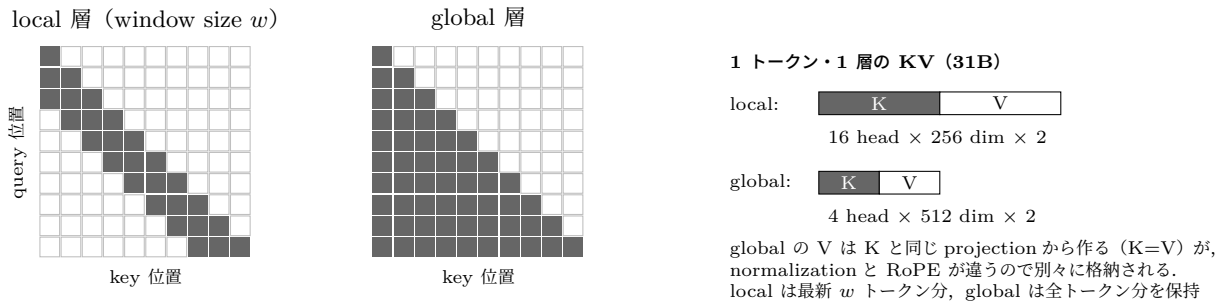


図 7: local 層と global 層の Attention mask（左・中，灰色が参照する key）と，31B で 1 トークン・1 層が持つ KV（右，帯の長さが要素数に比例）．

推論への効果は**格納量と読み出し量で異なる**．KV キャッシュに保存されるのは 15 層分だけなので，32k トークンのリクエスト 1 本の KV は 462 MiB から 198 MiB に減り，同時に載せられるリクエスト数が増える．一方，共有層も Attention 計算のたびに参照先の KV を読むので，decode 1 ステップの KV 読み出しは 35 層分のままである．つまり KV 共有は容量には効くが，長いコンテキストでの帯域は減らさない．

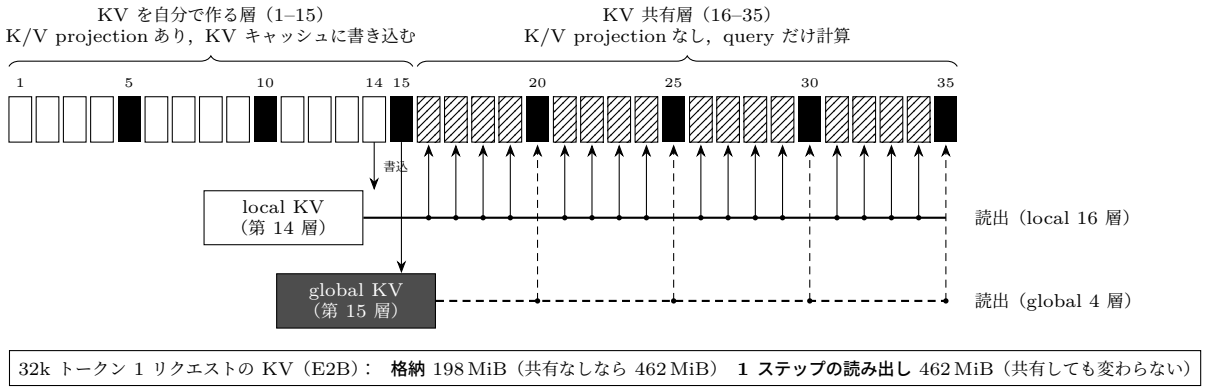


図 8: E2B の KV 共有．第 16–35 層は KV を作らず，第 14 層（local）と第 15 層（global）が保存した KV を読む．白は local 層，黒は global 層，斜線は KV 共有の local 層．

コード 2: vLLM の KV 共有の実装（抜粋）．参照先は「共有区間より前にある，同じ種類の最後の層」である．

```
# vllm/model_executor/models/gemma4.py  class Gemma4Attention (excerpt)
first_kv_shared_layer_idx = config.num_hidden_layers - num_kv_shared_layers
self.is_kv_shared_layer = (
    num_kv_shared_layers > 0 and layer_idx >= first_kv_shared_layer_idx
)
if self.is_kv_shared_layer:
    # K/V come from the target layer's cache, so like HF this layer
    # only has q_proj (and no k_norm/v_norm).
    self.q_proj = ColumnParallelLinear(hidden_size, self.total_num_heads * self.head_dim, ...)
...
# Find the last non-shared layer of the same attention type
prev_layers = config.layer_types[:first_kv_shared_layer_idx]
current_layer_type = config.layer_types[layer_idx]
kv_shared_layer_index = (
    len(prev_layers) - 1 - prev_layers[::-1].index(current_layer_type)
)                                          # <- E2B: local -> layer 14, global -> layer 15

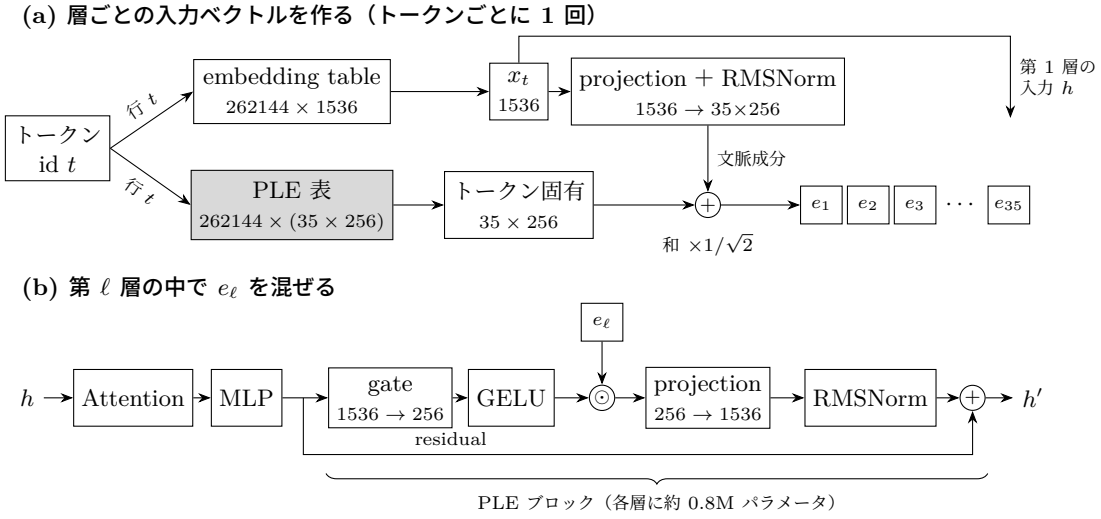
def forward(self, positions, hidden_states, **kwargs):
    if self.is_kv_shared_layer:
        # Shared KV: only Q is projected; K/V come from the target layer.
        q, _ = self.q_proj(hidden_states)
        ...
        attn_output = self.attn(q, None, None)  # <- nothing is written to the KV cache
```

5.3 Per-Layer Embeddings（PLE，E2B/E4B）

通常の embedding は，トークン id から 1 本のベクトルを引いて第 1 層に入れるだけである．PLE ではこれに加えて，**層ごとに**専用の小さなベクトル e_ℓ（256 次元）を各トークンに与える（図 9）．e_ℓ は，PLE 表から引いたトークン固有の成分と，通常の embedding x_t を projection した文脈成分の和である．各層は Attention と MLP の後に，hidden を 256 次元に絞った gate と e_ℓ の element-wise product をとり，元の次元に戻して residual に足す（コード 3）．

PLE 表は $262144 \times (35 \times 256)$ と巨大で，E2B の総 5.1B パラメータのうち 2.35B を占める．しかし 1 トークンで使うのはその 1 行（17.9 KB）だけで，本体の重みのように毎ステップ全体を読む必要はない．そのため E2B が decode で毎ステップ読む重みは，本体と LM head を合わせた 2.3B 分（4.6 GB）に収まる．図 1 で E2B/E4B の「読むパラメータ」が総数の約半分なのはこのためである．

一方，Gemma 4 は入力 embedding と LM head を共有しており，ロジット計算では 262144×1536 の行列全体を毎ステップ読む．E2B ではこの LM head が decode の読み出しバイトの 17% を占める．



decode 1 トークンで読むバイト：PLE 表は全体で 4.7 GB あるが，引くのは行 t の $35 \times 256 \times 2\,\mathrm{B} = 17.9\,\mathrm{KB}$ だけ．本体の重み（3.8 GB）や LM head（0.8 GB）のように全体を毎ステップ読む必要はない．
⇒ 総 5.1B パラメータのうち，帯域を消費するのは約 2.3B 分だけになる．

図 9: E2B の Per-Layer Embeddings．(a) トークンごとに 35 層分の入力 $e_1, \ldots, e_{35}$ を作り，(b) 第 ℓ 層は e_ℓ を gate 付きで residual に足す．(c) PLE 表は大きいが，decode で読むのは 1 行だけである．

5.4 MoE（26B-A4B）

各層で 128 個の routed expert（中間次元 704）から top-8 を選び，常に通る dense MLP（中間次元 2112）と足し合わせる（図 10，コード 4）．1 トークンが読む重みは約 3.8B（7.65 GB）で，総 25.2B の 15% である．ただしバッチ内の各トークンは別々の expert を選ぶので，B トークンのバッチで読む expert の割合は増えていく（図 10 右，式は §6.1）．**この割合は，各トークンが 128 個の expert を等しい確率で，ほかのトークンと独立に選ぶと仮定した理論値である．**実際のルーティングには偏りがあり，同じ expert に選択が集中すれば，読む割合はこれより小さくなる．本書ではルーティングの実測はしていない．

コード 3: vLLM の PLE の実装（抜粋）．(a) は図 9(a)，(b) は図 9(b) に対応する．PLE 表は `embed_tokens_per_layer` の行を引くだけである．

```python
# vllm/model_executor/models/gemma4.py  (excerpt)
# (a) once per token: build e_1..e_L
def get_per_layer_inputs(self, input_ids):
    per_layer_embeds = self.embed_tokens_per_layer(per_layer_inputs_tokens)  # <- row lookup only
    per_layer_embeds = per_layer_embeds * self.embed_scale_per_layer
    return per_layer_embeds.reshape(*input_ids.shape, self.config.num_hidden_layers,
                                    self.hidden_size_per_layer_input)

def project_per_layer_inputs(self, inputs_embeds, per_layer_inputs):
    per_layer_projection = self.per_layer_model_projection(inputs_embeds)
    per_layer_projection = per_layer_projection * self.per_layer_projection_scale
    per_layer_projection = per_layer_projection.reshape(...)
    per_layer_projection = self.per_layer_projection_norm(per_layer_projection)
    return (per_layer_projection + per_layer_inputs) * self.per_layer_input_scale  # <- 1/sqrt(2)

# (b) class Gemma4DecoderLayer.forward: after attention and MLP
if per_layer_input is not None and self.per_layer_input_gate is not None:
    gate = self.per_layer_input_gate(hidden_states)           # <- 1536 -> 256
    gate = torch.nn.functional.gelu(gate, approximate="tanh")
    gated_per_layer = gate * per_layer_input                  # <- elementwise with e_l
    per_layer_contribution = self.per_layer_projection(gated_per_layer)  # <- 256 -> 1536
    per_layer_contribution = self.post_per_layer_input_norm(per_layer_contribution)
    hidden_states = hidden_states + per_layer_contribution
```

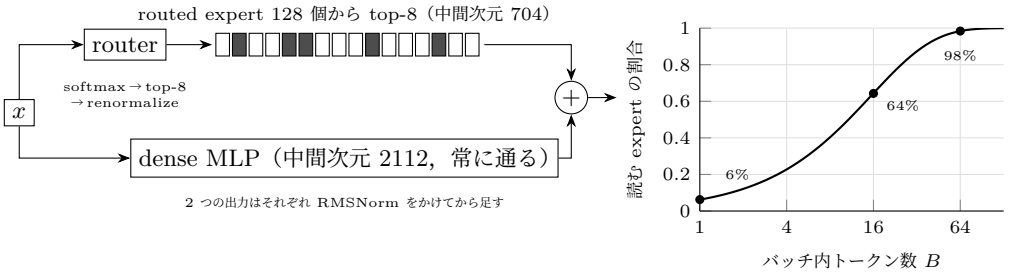


図 10: 26B-A4B の MoE ブロック（左）と，バッチ内トークン数 B に対して 1 ステップで読む routed expert の割合 $1-(1-8/128)^B$（右）．右は全 expert が等確率・独立に選ばれると仮定した理論値．

コード 4: vLLM の MoE ブロック（抜粋）．dense MLP と expert は同じ層の入力から並列に計算され，それぞれ normalization してから足される．

```python
# vllm/model_executor/models/gemma4.py  class Gemma4DecoderLayer.forward (excerpt)
# MLP runs unconditionally (same inputs for MoE and non-MoE)
hidden_states = self.pre_feedforward_layernorm(hidden_states)
hidden_states = self.mlp(hidden_states)                       # <- dense MLP (2112)
if self.enable_moe_block:
    hidden_states_1 = self.post_feedforward_layernorm_1(hidden_states)
    hidden_states_2 = self.pre_feedforward_layernorm_2(residual)
    router_logits = self.router(residual)                      # <- logits over 128 experts
    hidden_states_2 = self.experts(hidden_states_2, router_logits)  # <- fused MoE, top-8
    hidden_states_2 = self.post_feedforward_layernorm_2(hidden_states_2)
    # Combine MLP and MoE outputs
    hidden_states = hidden_states_1 + hidden_states_2

# class Gemma4Router
def routing_function(self, hidden_states, gating_output, topk, renormalize):
    """Softmax over ALL experts -> top-k -> renormalize."""
    return gemma4_fused_routing_kernel_triton(gating_output, topk, self.per_expert_scale)
```

5.5 MTP Drafter

MTP Drafter は 4 層（local 3 + global 1）の小型 Transformer で，target の最終層の hidden state と直前トークンの embedding を連結し，projection して入力にする（図 11）．2 トークン目以降の draft では，target の hidden の代わりに drafter 自身の出力を target の次元に projection し直したものを使う．drafter は自前の KV キャッシュを持たない．local 層は target の最後の local 層，global 層は最後の global 層の KV を参照する．vLLM の実装では，γ 回の draft はすべて同じ位置（target が最後に処理した位置）で計算され，draft したトークンは KV に書き込まれない．drafter が自分の draft を知る手がかりは，戻される hidden とトークンの embedding だけである（コード 5）．このため drafter 側の prefill が不要で，draft length γ を自由に選べる．E2B/E4B の drafter は，vocabulary 262k 全体ではなく，2048 個のクラスタ centroid から選んだ top-32 クラスタ（約 4k 語）だけでロジットを計算する（コード 6）．drafter が 1 回に読むバイト数は target の 0.3–11% である（表 6）．公式は最大 3 倍の高速化をうたい，vLLM の実装 PR[6] は H100・同時実行 1 で 31B（$\gamma = 8$）3.19 倍，26B-A4B（$\gamma = 4$）1.60 倍を報告している．

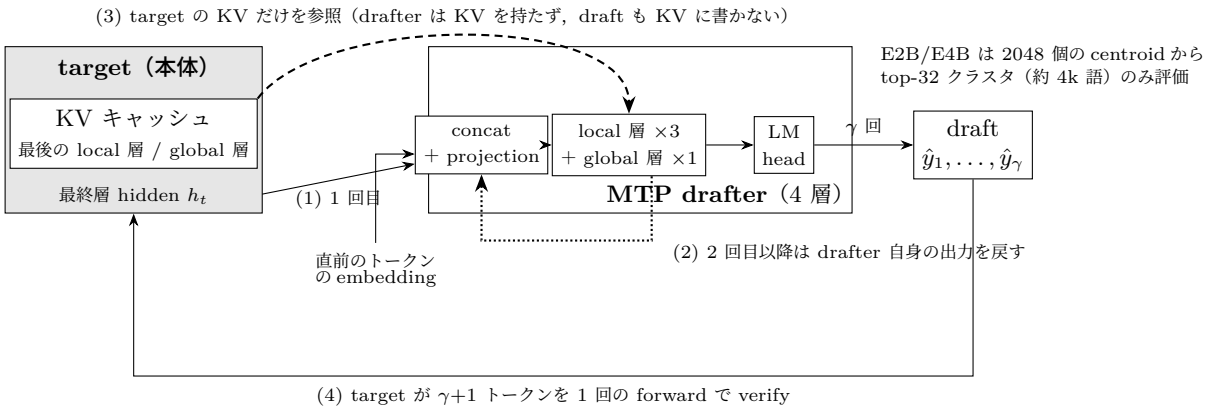


図 11: vLLM 実装に基づく MTP Drafter の構造．(1) 1 回目は target の最終層 hidden と直前トークンの embedding を入力にし，(2) 2 回目以降は drafter 自身の出力を target の次元に戻して入力にする．(3) Attention は target の最後の local 層・global 層の KV だけを参照し，(4) target が $\gamma + 1$ トークンを 1 回の forward で verify する．

コード 5: vLLM の MTP Drafter（抜粋）．(1)–(3) は図 11 の番号に対応する．

```
# vllm/model_executor/models/gemma4_mtp.py  class Gemma4MultiTokenPredictor (excerpt)
def forward(self, input_ids, positions, hidden_states, ...):
    """Returns (draft_hidden_states, backbone_hidden_states).
    backbone_hidden_states: backbone-dim, stored in the proposer's
        hidden-state buffer and fed back as input to the next step.
    """
    inputs_embeds = self.embed_input_ids(input_ids)
    combined = torch.cat([inputs_embeds, hidden_states], dim=-1)   # <- (1) embedding + hidden
    hidden_states, _ = self.pre_projection(combined)               # <- 2 x backbone -> draft dim
    for layer in self.layers:                                      # <- 4 Q-only layers
        hidden_states, residual = layer(positions=positions, hidden_states=hidden_states, ...)
    draft_hidden_states = self.norm(hidden_states)                 # <- -> LM head (draft dim)
    backbone_hidden_states, _ = self.post_projection(draft_hidden_states)  # <- (2) fed back
    return draft_hidden_states, backbone_hidden_states

# vllm/v1/spec_decode/gemma4.py  (excerpt)
# All draft steps predict from the same position (the last
# target-model position), so positions and seq_lens must not
# advance between steps.
self.constant_draft_positions = True
...
# Each draft decoder layer is mapped to the last non-KV-shared
# target layer of the same attention type (sliding or full).
target_idx = candidates[-1]                                        # <- (3)
attn.kv_sharing_target_layer_name = f"{target_prefix}.{target_idx}.self_attn.attn"
```

コード 6: E2B/E4B の drafter の centroid ヘッド（抜粋）．LM head 全体ではなく，選んだ 4096 語の行だけを読む．

```
# vllm/model_executor/models/gemma4_mtp.py  class Gemma4MTPMaskedEmbedder (excerpt)
def _select_and_score(self, hidden_states, lm_head_weight):
    _, top_k_indices = torch.topk(self.centroids(hidden_states),     # <- score 2048 centroids
                                  k=self.centroid_intermediate_top_k, dim=-1)  # <- keep 32
    clusters = self.token_ordering.view(self.num_centroids, self.vocab_size_per_centroid)
    selected = clusters[top_k_indices]                                # <- 32 x 128 = 4096 tokens
    embeddings = lm_head_weight[selected.reshape(-1)].view(
        num_tokens, self.num_selected, self.hidden_size)              # <- read only these rows
    logits = torch.einsum("td,tsd->ts", hidden_states, embeddings)
```

表 4: Gemma 4 の工夫と，それが効くはずの局面．

工夫	減らすもの	効くはずの局面	対象
Hybrid Attention	KV 量，prefill の Attention FLOPs	長いコンテキスト	全モデル
global 層の K=V	V projection の重みと計算	–（KV 量は現行実装では減らない）	12B / 26B / 31B
KV 共有	KV の格納量（読み出しは減らない）	高並列（KV 容量）	E2B / E4B
PLE	decode で毎ステップ読む重み	decode 全般	E2B / E4B
MoE	decode で読む重み	低負荷の decode	26B-A4B
MTP Drafter	target の forward 回数	低負荷の decode	全モデル

6 理論モデル

§3.2 の roofline を decode 1 ステップと prefill に当てはめ，必要なバイト数 Q と FLOPs F をチェックポイントから正確に数える．重みは safetensors のヘッダから{本体，routed expert，embedding/LM head，PLE テーブル，視覚・音声エンコーダ}に分類し（エンコーダはテキスト生成では使わないので除外），KV は `config.json` の層種別・window size・KV ヘッド数・KV 共有から層ごとに計算する．

■記号． W_{body}：本体の重み（Attention の projection，dense MLP，normalization）のバイト数，W_{LM}：LM head のバイト数，W_{exp}：routed expert 全体のバイト数，E：expert 数，k：1 トークンが選ぶ expert 数．層 ℓ について，w_ℓ：window size（global 層は ∞），h_ℓ^q：query ヘッド数，d_ℓ：head dim，$b_\ell = 2 \cdot h_\ell^{kv} \cdot d_\ell \cdot 2\,\text{byte}$：1 トークンあたりの KV バイト数（K と V の 2 つ．K=V の層も，実装では K と V を別々に格納するので 2 とする）．$\mathcal{L}$，$\mathcal{G}$：local 層と global 層の集合（KV 共有層も含む）．

6.1 Decode の読み出しバイト

同時実行数 B，コンテキスト長 L の decode 1 ステップで HBM から読むバイト数を

$$Q(B,L) = W_{\text{body}} + W_{\text{LM}} + W_{\text{exp}}\, f(B) + B\Big(\underbrace{\sum_{\ell\in\mathcal{L}} b_\ell \min(L,w)}_{\text{local 層}} + \underbrace{\sum_{\ell\in\mathcal{G}} b_\ell\, L}_{\text{global 層}}\Big), \qquad f(B) = 1 - \Big(1 - \frac{k}{E}\Big)^B \tag{4}$$

とする．各項の根拠は次のとおりである．

- **本体の重み** W_{body}：B 本の系列は 1 つの GEMM にまとめて計算されるので，重みはバッチ全体で 1 回読めばよく，B によらない．重みは数 GB 以上あり L2（60 MiB）には載らないので，毎ステップ HBM から読み直す．
- **LM head** W_{LM}：次トークンの分布を出すには vocabulary 262,144 語すべてのロジットが必要で，LM head 全体を毎ステップ読む．入力側の embedding と PLE テーブルは B 行を引くだけなので無視できる．

- **routed expert** $W_{\mathbf{exp}} f(B)$：選ばれなかった expert は読まない．各トークンが E 個から k 個を独立・一様に選ぶと仮定すると，ある expert が B トークンの誰にも選ばれない確率は $(1-k/E)^B$ なので，読む expert の割合の期待値は $f(B)$ になる．$B=1$ では k/E（26B-A4B で 6.3%），B が大きいと 1 に近づく．実際のルーティングには偏りがあり，トークン同士が同じ expert を選びやすければ実際の読み出しはこれより少ない．本書で MoE の読み出しバイトとして示す値（表 6，§6.5，§8）は，すべてこの一様・独立の仮定による理論値である．
- **KV**：重みと違って KV は系列ごとに別なので B 倍になる．**local 層と global 層は，参照するトークン数と 1 トークンあたりの KV の両方が違う**ので分けて足す（表 5）．local 層は直近 $\min(L,w)$ トークンしか参照しないので，$L>w$ では L によらない定数になる．global 層は全 L トークンを参照するので L に比例するが，KV ヘッドが少ない（1–4 個）ので b_ℓ は小さい．KV 共有層は KV を格納しないが読み出しは行うので，どちらの和も共有層を含む全層についてとる．同じ和を KV を持つ層だけについてとったものが，KV の格納量（§6.4）になる．

表 5: 式 (4) の KV 項の中身（BF16，1 系列あたり）．30k は $L=30{,}000$ での 1 ステップの読み出し量．

モデル	層	層数	b_ℓ（KV / トークン）	参照するトークン数	30k で 1 層	30k で全層
Gemma4 E2B	local	28	1 KiB（1 head × 256）	$\min(L,512)$	0.5 MiB	14 MiB
	global	7	2 KiB（1 head × 512）	L	59 MiB	410 MiB
Gemma4 31B	local	50	16 KiB（16 head × 256）	$\min(L,1024)$	16 MiB	800 MiB
	global	10	8 KiB（4 head × 512）	L	234 MiB	2.3 GiB
Qwen3-32B	global	64	4 KiB（8 head × 128）	L	117 MiB	7.3 GiB

6.2 Decode の FLOPs とステップ時間の下限

$$F(B,L) = B\Big(2N_{\text{active}} + \sum_{\ell} 4\,h_\ell^q\, d_\ell \min(L, w_\ell)\Big) \tag{5}$$

- **線形層**：1 トークンにつき，使われるパラメータ（本体＋選ばれた expert ＋ LM head，計 N_{active} 個）を 1 回ずつ積和に使うので $2N_{\text{active}}$ FLOP.
- **Attention**：query ヘッド 1 個につき，$\min(L,w_\ell)$ 個の key との内積（$QK^\top$）に $2d_\ell \min(L,w_\ell)$ FLOP，その重みで value を足し合わせる（PV）のに同じだけかかり，計 $4d_\ell \min(L,w_\ell)$ FLOP．GQA は KV ヘッドを減らすが query ヘッドは減らさないので，h_ℓ^q 倍する．

式 (1) から，ステップ時間の下限は $t_{\text{step}} \geq \max(Q/\beta,\ F/\pi)$ である．本書の条件（$B \leq 64$）では $I = F/Q \lesssim 64 < I^*$ なので，常にバイト数の項が効き，$\text{TPOT}_{\min} = t_{\text{step}}(B{=}1) \approx Q(1,L)/\beta$ となる．実測の TPOT からは **HBM 利用率** $= \big(Q/\text{TPOT}\big)/\beta$ を求め，理論帯域をどれだけ使えているかの指標にする．

6.3 Prefill の FLOPs

プロンプト長 P に対し

$$F_{\text{prefill}}(P) = 2PN_{\text{body}} + \sum_{\ell} 4h_\ell^q d_\ell \sum_{p=1}^{P} \min(p, w_\ell), \qquad \text{TTFT} \geq F_{\text{prefill}}/\pi \tag{6}$$

とする（N_{body} は LM head を除いた 1 トークンあたりの使用パラメータ数）．

- 線形層は P トークンそれぞれに $2N_{\text{body}}$ FLOP かかる．LM head は最後のトークンにしか使わないので無視できる．

- 因果的 Attention では，p 番目のトークンは自分以前の p トークン（local 層では最大 w_ℓ トークン）を参照するので，内側の和 $\sum_p \min(p, w_\ell)$ になる．これは global 層で $P(P+1)/2$（P^2 に比例），local 層で $w(w+1)/2 + (P-w)w$（P に比例）である．**長いプロンプトで Hybrid Attention が prefill を速くするのは，P^2 で増える項が global 層だけに限られるから**である．
- prefill は $I \approx P \gg I^*$ なので計算律速とみなし，FLOPs を演算ピークで割って下限とする．実効 FLOP レート $F_{\text{prefill}}/\text{TTFT}$ を π で割った値を **prefill MFU**（Model FLOPs Utilization）と呼ぶ．MFU は PaLM[20] が導入した指標で，実装の都合で追加される計算（再計算など）を含めず，モデルが本来必要とする FLOPs だけを数えて「ピーク性能のうちどれだけを有効な計算に使えたか」を表す．

6.4 KV キャッシュの格納量

1 リクエストが格納する KV は $\sum_{\ell \in \text{KV を持つ層}} b_\ell \min(L, w_\ell)$ である．GPU 上の KV 領域をこの値で割ると，長さ L のリクエストを最大何本同時に処理できるかが分かる．

6.5 speculative decoding のステップコスト

式 (3) のコスト c を，帯域律速を仮定して

$$c_{\text{pred}}(\gamma) = \frac{Q_{\text{target}}(B{=}\gamma{+}1) + \gamma\, Q_{\text{draft}}}{Q_{\text{target}}(B{=}1)} \tag{7}$$

と見積もる．

- **verify**：$\gamma+1$ 位置を 1 回の forward で処理するので，重みの読み出しは $\gamma+1$ トークンのバッチと同じである．dense モデルでは式 (4) の重みの項が B によらないので，$Q_{\text{target}}(\gamma{+}1) \approx Q_{\text{target}}(1)$，つまり verify はほぼ無料である．MoE では $f(\gamma{+}1)$ が $f(1)$ より大きいので，verify のたびに読む expert が増える．
- **draft**：drafter を γ 回逐次に動かし，毎回その重み Q_{draft}（E2B/E4B では centroid ヘッドで選んだ部分の LM head を含む）を読む．
- verify の arithmetic intensity は $\gamma + 1 \le 9 \ll I^*$ なので帯域律速の仮定は成り立つ．カーネル起動などの固定費は含まないので，c_{pred} は実際の c の下限である．

6.6 理論値の一覧

式 (4)–(7) から求めた理論値を表 6 に，1 トークンあたりの KV 格納量を図 12 に示す．

表 6: 理論値（BF16，H100 NVL）．GB/step と TPOT_{min} は $B = 1$, $L = 2048$．KV@32k は 32k トークンのリクエスト 1 本の KV 格納量．

Model	active [B]	text total [B]	GB/step	TPOT_{min} [ms]	tok/s_{max}	KV@32k [GiB]	TTFT_{min}@4k [ms]	draft/target bytes
Gemma4 E2B	2.30	4.6	4.6	1.19	840	0.19	20	0.6%
Gemma4 E4B	4.70	7.5	9.4	2.43	411	0.52	41	0.3%
Gemma4 12B	11.94	11.9	23.9	6.22	161	0.81	113	3.5%
Gemma4 26B-A4B	3.82	25.2	7.6	2.02	494	0.82	34	11.0%
Gemma4 31B	30.70	30.7	61.4	16.00	62	3.28	301	1.5%
Qwen3-30B-A3B	3.04	30.5	6.1	1.61	621	3.00	35	–
Qwen3-32B	31.98	32.8	64.0	16.54	60	8.00	327	–

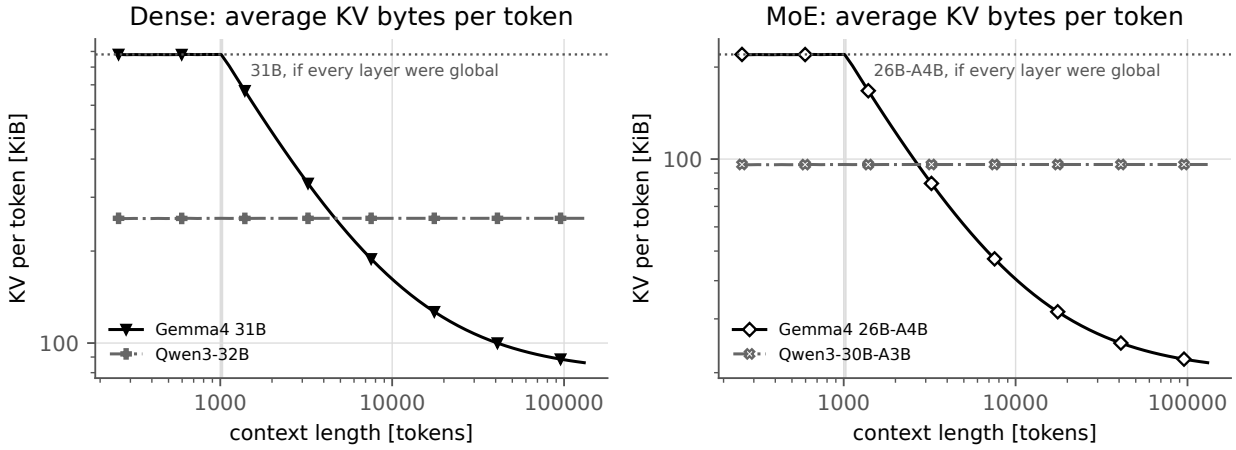


図 12: 1 トークンあたりの平均 KV 量．同クラスの Gemma 4 と Qwen3 の比較で，点線は Gemma 4 の全層を global とした場合．以降のグラフはすべて，マーカーの形でモデルを，塗りつぶしで dense（塗り）/MoE（白抜き）を，線種で Gemma 4（実線）/Qwen3（一点鎖線）を区別する．window size（1024）を超えると local 層の KV が頭打ちになり，32k では全層 global の場合の約 1/8 に下がる．一方，window size 以下の短いコンテキストでは local 層の KV が大きく，31B は約 4.7k トークン未満で Qwen3-32B より，26B-A4B は約 2.7k トークン未満で Qwen3-30B-A3B より KV が大きい．

7　実験方法

表 7 の 9 つの実験を行った．実験番号は付属リポジトリの `README.md` と対応しており，実行コマンドとオプションの詳細はそちらとソースコードを参照されたい．

表 7: 実験の一覧．

番号	実験	目的	結果
E1	単一ストリーム	同時実行 1 での TTFT・TPOT をプロンプト長ごとに測り，理論下限と比べる	図 13–19，表 9
E2	同時実行数スイープ	負荷を上げたときのスループットと遅延，帯域の償却を見る	図 22–25
E3	speculative decoding	MTP の γ と生成内容による AL・高速化の変化，n-gram との比較	図 26–31，表 16
E4	KV 容量	vLLM が確保する KV 領域と同時実行数の上限を理論値と比べる	表 15
E5	マイクロベンチ	本機で実際に出る GEMM 性能と HBM 帯域を測る	表 2
E6	Nsight Systems	decode 1 ステップの時間をカーネル種別に分解する	図 17，20，29，表 10
E7	Nsight Compute	長コンテキストの Attention カーネルの律速要因を特定する	表 13
E8	追加計測	Attention backend（Triton）と FP8 KV キャッシュで E1/E2 を測り直し，原因を確かめる．26B-A4B の MTP と E5 の再現性を確かめる	表 2，14，§8.5
E9	カーネル解析と修正	decode の Attention・GEMM カーネルを単体で測り，split-KV を修正した FA4 で E1・E2（31B）・E6 を測り直す	図 18，21，表 11，12，§8.4

■サーバと計測（E1–E4）． 各構成で `vllm serve` を起動し（BF16，CUDA Graph 有効，最大同時系列数 128，最大長 40,960．31B のみ重みが 58 GiB と大きいため 34,816），クライアントから OpenAI 互換 API にストリーミングでリクエストを送った．同じプロンプトを繰り返し送るので，TTFT がキャッシュで短くならないよう prefix caching は無効にした．TTFT は最初のトークンが届くまでの時間，TPOT は（最後のトークンの時刻 − 最初のトークンの時刻）/（出力トークン数 −1）である．speculative decoding の AL は，`/metrics` の draft 数と accepted token 数をリクエストの前後で差分して求めた．なお，オフライン API（`LLM.generate`）の内部タイムスタンプから求めた TTFT は物理的な下限を下回ったため使っていない．

■ワークロード． 実データとして Babilong と SWE-bench Lite を使い，条件を制御するために合成プロンプトも加えた（表 8）．greedy 生成なので，speculative decoding の有無で出力は同一であり，同じ出力の生成速度を比較できる．

表 8: ワークロード．

名前	内容	ねらい
Synthetic	プロンプト約 256–16k トークン，出力 128 トークン固定，各 3 回	長さを細かく制御した掃引
Babilong[9]	長さ 1k–32k × タスク qa1–qa3 × 2 件，出力 128 トークン固定	実データでの長コンテキスト
SWE-bench Lite[10]	8 リポジトリ × 3 件，パッチ生成（最大 512 トークン）	自然な生成，ジャンル別の acceptance rate

同時実行数スイープ（E2）では SWE-bench のプロンプト（平均約 1.3k トークン）を巡回させ，出力 256 トークン固定で同時実行数 $1, 2, \ldots, 64$ を測った．

■プロファイリング（E5–E7）． Nsight Systems では CUDA Graph 内のカーネルもノード単位で記録し，カーネル名から GEMM・MoE・Attention・normalization などに分類した．Nsight Compute では，E2B の約 30k トークン後の decode 1 ステップぶん（Attention カーネル 35 回）のカウンタを取得した．

8 結果

表 9 に単一ストリームの代表値と，同時実行スイープでの最大スループットをまとめる．

表 9: 実測サマリ．TTFT は約 1k と 32k トークンのプロンプト（ms）．prefill MFU（Model FLOPs Utilization）は 8k トークン超のプロンプトでの（モデル FLOPs / TTFT）/ 835 TFLOPS．TPOT は 2k トークン未満のプロンプトの中央値（ms），TPOT_{min} は同条件の理論下限，HBM util. は（理論読み出しバイト / TPOT）/ 3.9 TB/s．MTP 列は $\gamma = 4$ の TPOT 高速化（中央値）と mean acceptance length AL（Synthetic + SWE-bench）．max out tok/s は同時実行スイープでの最大出力スループット．

Model	TTFT 1k	TTFT 32k	prefill MFU	TPOT	TPOT_{min}	HBM util.	MTP speedup	AL	max out tok/s
Gemma4 E2B	22	602	36%	4.21	1.19	28%	1.94×	2.78	6869
Gemma4 E4B	25	896	43%	6.23	2.42	39%	2.17×	2.90	4264
Gemma4 12B	42	2203	47%	10.41	6.19	59%	1.85×	3.20	1659
Gemma4 26B-A4B	35	1051	33%	5.30	2.00	38%	1.35×	3.46	2392
Gemma4 31B	104	5999	45%	22.87	15.91	70%	3.08×	3.58	533
Qwen3-30B-A3B	45	1563	41%	4.56	1.58	35%	–	–	2599
Qwen3-32B	106	7179	49%	22.01	16.45	75%	–	–	839

8.1 Prefill：Hybrid Attention は長いプロンプトで効く

■大きい dense モデルは本機の GEMM 性能をほぼ使い切っている． 2k トークン以上の実効 FLOP レートは 12B/31B/Qwen3-32B で 380–415 TFLOPS にそろう（図 13）．データシート値の 45–49% だが，本機の実測 GEMM ピーク（492–514 TFLOPS，表 2）に対しては 74–84% に当たる．一方 MoE は 8k トークン超でも 26B-A4B が 265–277 TFLOPS，Qwen3-30B-A3B が 305–356 TFLOPS にとどまる．expert ごとの GEMM（中間次元 704 など）は小さく，プロンプトのトークンが 128 個の expert に分散するため，1 つの GEMM あたりの行数も少なくなるからである．

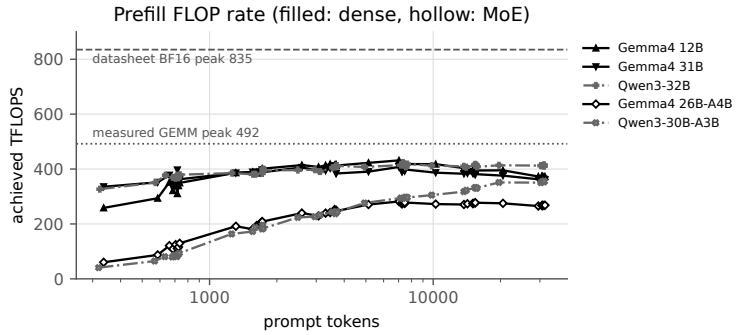


図 13: prefill の実効 FLOP レート（モデル FLOPs / TTFT）．塗りは dense，白抜きは MoE．破線はデータシート値，点線は本機の実測 GEMM ピーク（初回の計測値 492 TFLOPS）．

■Hybrid Attention は長いプロンプトで効く． sliding window の効果を確かめるため，同クラスの Qwen3 に対する TTFT の比を，式 (6) の FLOPs の比と並べた（図 14）．点線は Gemma 4 の全層を global とした場合の FLOPs の比で，実線との差が sliding window で減った分である．

- **window size（1024）以下では差がない．**このとき local 層も全トークンを参照するので，Attention の FLOPs は Gemma 4 と全層 global で同じになる．dense の組では TTFT の比は 0.95–0.98 で，パラメータ数の差による FLOPs の比（0.94）とほぼ一致する．
- **効果が見えるのは 4k トークン以上である．**window size を超えても，Attention が prefill の FLOPs に占める割合は 2k トークンで 3% 程度と小さく，sliding window で削れる量も小さい．P^2 で増える Attention の割合が大きくなる 4k 以上で，実線と点線が離れ，測った TTFT の比も下がっていく．
- **32k では 31B が 16%，26B-A4B が 33% 速い．**TTFT は 31B が 6.0 s 対 7.2 s，26B-A4B が 1.05 s 対 1.56 s である．Attention が FLOPs に占める割合は，Qwen3-32B の 36% に対し 31B は 18% である．

ただし 32k での実測の比（0.84，0.68）は FLOPs の比（0.73，0.51）ほど下がらない．31B の実効 FLOP レートは 32k で Qwen3-32B より 1 割ほど低く（図 13），Gemma 4 の Attention 計算（head dim 512 を含む FA4 系カーネル）の効率が Qwen3 の FlashAttention 3 より低い可能性がある．また 26B-A4B は window size 以下の短いプロンプトで FLOPs が多いのに Qwen3-30B-A3B より 15–20% 速い．この領域の TTFT は 35–50 ms で計算量では決まっておらず，Attention とは別の要因である．

8.2 短いコンテキストの decode：大きいモデルほど理論値に近い

図 15 のとおり，1 ステップで読むバイト数が大きいモデルほど理論帯域に近い．

理由を調べるため，Nsight Systems で測った decode 1 ステップの時間を，「重みを実測帯域（3.57 TB/s）で読む理想時間」と，それを超える部分に分けた（図 16，表 10）．**理想時間を超える部分は，モデルの大きさによらず 1 ステップ 2.7–4.6 ms ある．**31B ではこれが理想時間 17.2 ms の 27% にすぎないが，E2B では理想時間

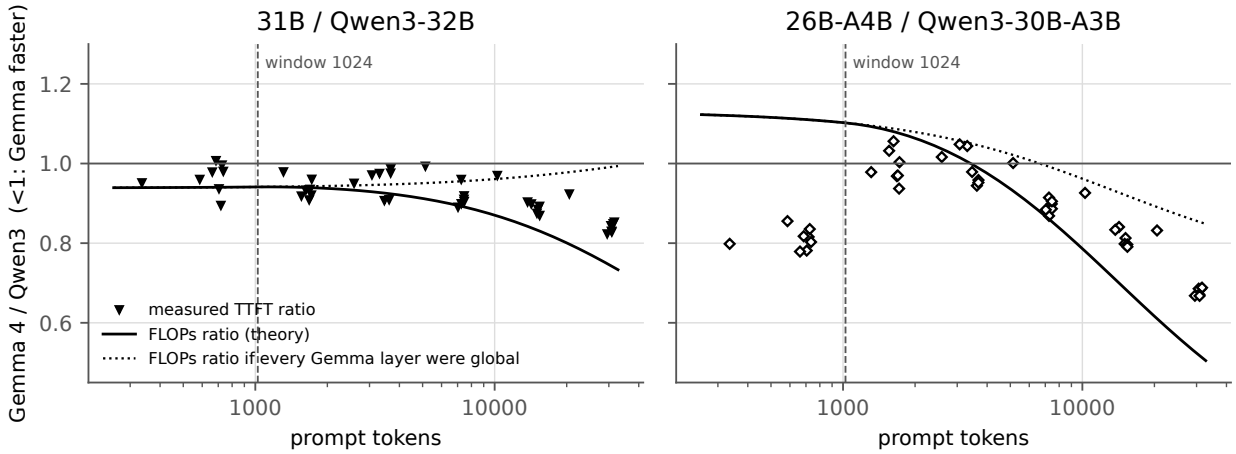


図 14: 同クラスの Qwen3 に対する TTFT の比（点）と prefill FLOPs の比（実線）．点線は Gemma 4 の全層を global とした場合の FLOPs の比．破線は window size 1024.

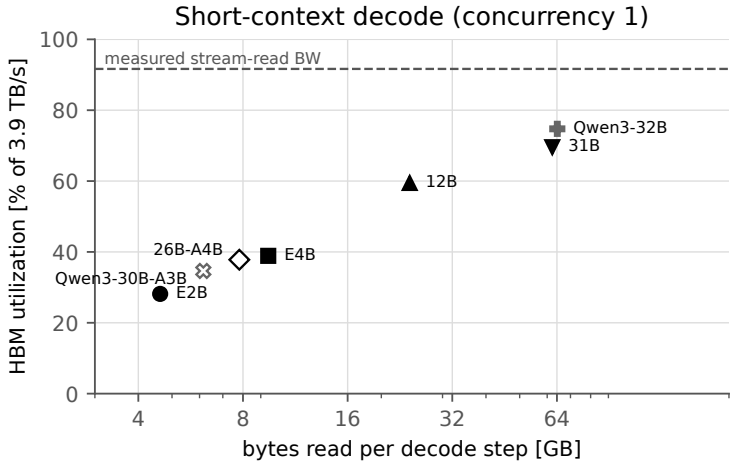


図 15: 短いコンテキスト（2k トークン未満）の decode における，1 ステップの読み出しバイト（式 (4)）と HBM 利用率.

1.3 ms の 2 倍になる．小さいモデルの帯域利用率が低いのは，この一定の上乗せが相対的に重いためである．

上乗せの中身は次のとおりで，いずれも「1 個あたり数 µs かかるカーネルが 1 ステップに数百個並ぶ」ことに由来する．CUDA Graph により CPU からのカーネル起動のコストはほぼ消えているが，GPU 上でカーネル 1 個ごとにかかる起動・終了の時間とカーネル間の隙間は残る．

表 10: decode 1 ステップの時間内訳（ms，バッチ 1，短いコンテキスト）．実効帯域 = 理論重みバイト / GEMM+MoE 時間.

	step	GEMM+MoE	実効帯域	attention	norm/elem.	idle
Gemma4 E2B	3.98	2.15	2.14 TB/s	0.61	0.74	0.40
Gemma4 12B	9.79	7.43	3.21 TB/s	0.86	1.03	0.41
Gemma4 26B-A4B	4.91	3.38	2.26 TB/s	0.52	0.56	0.40
Gemma4 31B	21.78	18.39	3.34 TB/s	1.19	1.60	0.55
Qwen3-30B-A3B	4.72	3.17	1.92 TB/s	0.73	0.34	0.44

- **31B の GEMM は帯域をほぼ使い切っている．**重み 61 GB を 18.4 ms で読み，3.34 TB/s（実測ピーク

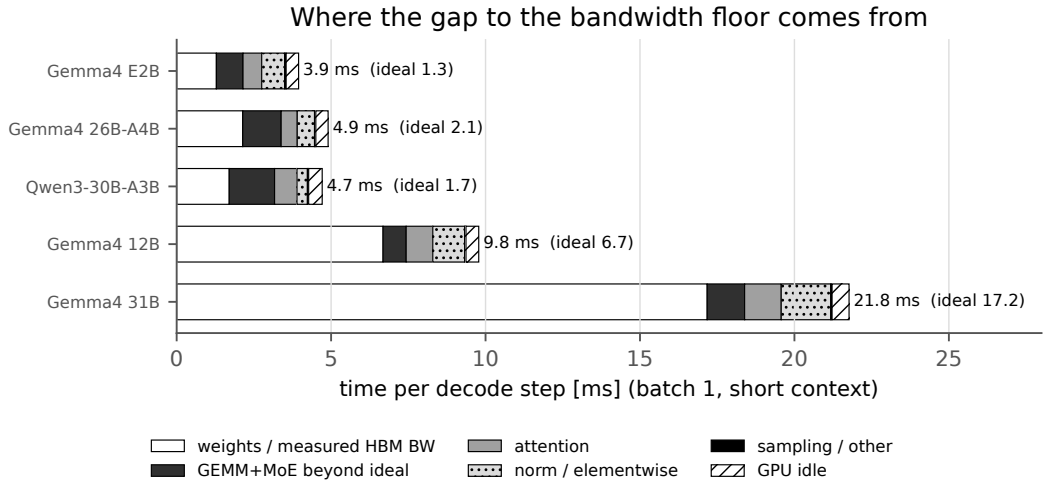


図 16: decode 1 ステップの時間の分解（Nsight Systems，バッチ 1，短いコンテキスト）．白は重みを実測 HBM 帯域で読む理想時間で，それ以外が帯域律速の下限からの差．

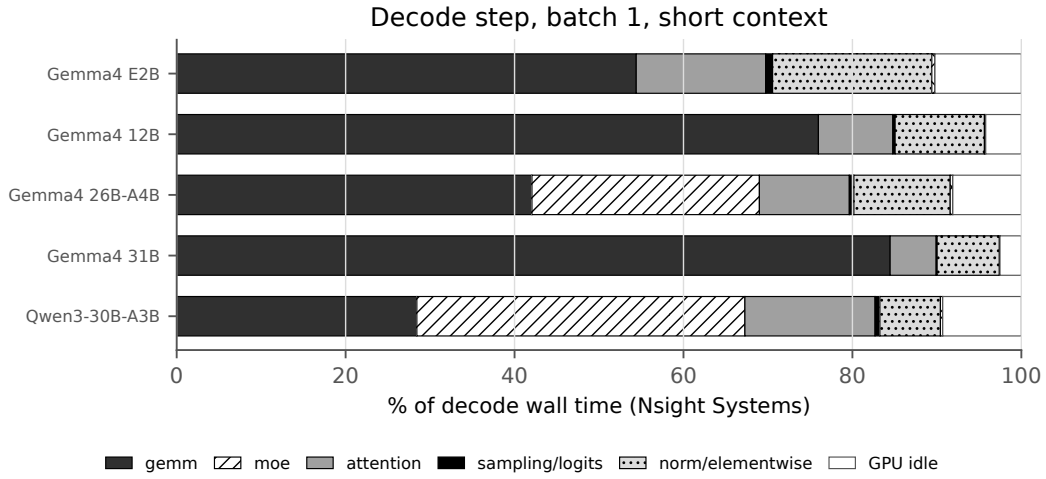


図 17: decode 1 ステップの時間内訳（Nsight Systems，バッチ 1，短いコンテキスト）．白は GPU idle.

の 94%）に達する．TPOT 全体では理論帯域の 70% で，残りは normalization など（1.6 ms），Attention（1.2 ms），idle（0.55 ms）である．

- **E2B は GEMM 自体も帯域を使い切れない．** 1 トークンあたり約 585 個のカーネルが走る．Nsight Systems によると，最も時間を使う GEMM カーネルは 1 回 6 µs，grid は平均 43 CTA である．§3.4 で見たとおり，132 SM のうち 89 個に仕事がなく，1 wave で終わってしまう（図 5(a)）．hidden 1536 の行列は小さく，CTA に分けても SM の数に届かないためで，GEMM は理想時間より 0.86 ms 長い．さらに RMSNorm や residual add などの小さなカーネルが 0.74 ms，35 層分の Attention が 0.60 ms，GPU idle が 0.40 ms かかる．
- **MoE の expert 計算は帯域効率が低い．** 26B-A4B と Qwen3-30B-A3B では GEMM+MoE が理想時間より 1.2–1.5 ms 長く，実効帯域は 1.9–2.3 TB/s にとどまる．expert 1 つあたりの行列（704×2816 など）が小さいためである．

■GEMM 1 個ごとの固定時間（E9）． この上乗せをハードウェアの側から確かめるため，decode と同じ $M = 1$ の GEMM を CUDA Graph の中で重みの大きさを変えて測った（図 18）．重みは L2（60 MiB）より大きな組を

巡回させ，毎回 HBM から読ませている．時間は $t \approx 3.8\,\mu\mathrm{s} + W/3.46\,\mathrm{TB/s}$（$W$ は重みのバイト数，$K = 1536$ で最小二乗）でよく近似できる．空のカーネルは 1.1 µs なので，残りの約 2.7 µs は最初のデータが HBM から届くまでの latency と，CTA が少なく帯域が立ち上がりきらない時間である．このため 6 MB の行列では 1.1 TB/s しか出ず，HBM 帯域に近づくには 1 回で 50 MB 以上を読む必要がある．1 ステップの GEMM の数は E2B が 212 個，12B が 193 個，31B が 241 個で，モデルの大きさによらずほぼ同じである．この式で 1 ステップの GEMM 時間を予測すると，E2B が 2.13 ms（実測 2.15 ms），12B が 7.6 ms（7.4 ms），31B が 18.7 ms（18.4 ms）で，3% 以内で一致する．固定分は E2B では 0.80 ms で重みの読み出し（1.33 ms）の 6 割に当たるが，31B では 0.91 ms で 17.7 ms の 5% にすぎない．**小さいモデルの帯域利用率が低いのは，約 200 個の GEMM がそれぞれ約 4 µs の固定時間を払い，読むバイト数が少ないほどその比率が大きくなるからである．**

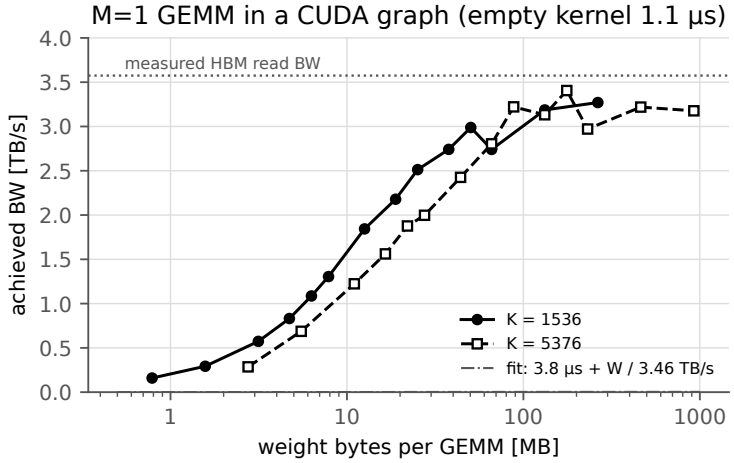


図 18: $M = 1$ の GEMM（CUDA Graph 内，重みは毎回 HBM から読む）の重みの大きさと達成帯域．一点鎖線は $K = 1536$ での $t = t_0 + W/\mathrm{BW}$ の当てはめ．

8.3 長いコンテキストの decode：Attention カーネルの split-KV のバグ

コンテキスト長に対する TPOT の増え方は，Gemma と Qwen で大きく異なる（図 19，表 11）．Qwen3 の増分は KV 読み出しの理論増分の 1.3–1.4 倍に収まるが，Gemma 4 は 26–120 倍である．理論下限（点線）はどちらもほぼ平らなので，この増分は KV の読み出し量では説明できない．以下で示すとおり，原因は vLLM 0.32.0 の FlashAttention-4 カーネルの split-KV のバグで，1 行の修正で増分はほぼ理論値まで下がる（表 11 の右 2 列）．

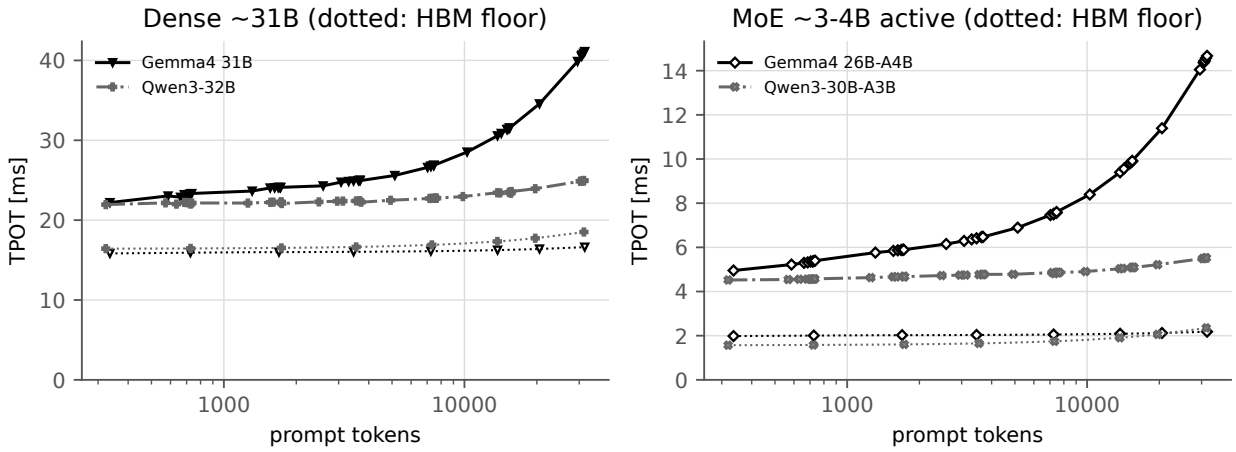


図 19: TPOT とプロンプト長．同クラスの Gemma 4 と Qwen3 の比較（点線は式 (4) による帯域律速の下限）．

表 11: TPOT のコンテキスト長依存性（ms）．理論増分は KV 読み出しの増分 / 3.9 TB/s．右 2 列は split-KV を修正した FA4（§8.4）で同じ条件を測り直した値（E9）．

	vLLM 0.32.0				split-KV 修正後	
モデル	TPOT（<1k）	TPOT（∼30k）	増分	理論増分	TPOT（∼30k）	増分
Gemma4 E2B	4.21	16.96	+12.8	+0.11	3.70	+0.11
Gemma4 E4B	6.23	19.02	+12.8	+0.22	5.79	+0.26
Gemma4 12B	10.41	23.96	+13.6	+0.15	9.93	+0.21
Gemma4 26B-A4B	5.30	14.44	+9.1	+0.17	4.87	+0.14
Gemma4 31B	22.96	40.66	+17.7	+0.69	23.27	+1.25
Qwen3-30B-A3B	4.56	5.52	+1.0	+0.77	–	–
Qwen3-32B	22.01	24.93	+2.9	+2.05	–	–

■どのカーネルが遅いか（Nsight Systems）． E2B の 30k トークン後の decode では，1 ステップ 19.0 ms の 79% を Attention カーネルが占める（図 20）．同じ FlashAttention-4 カーネルが local 層では 1 回 23 µs，global 層では 1 回 1.84 ms かかる．global 層 1 回で読む KV は約 61 MB なので，実効帯域は約 33 GB/s（HBM の 1%）にすぎない．31B（約 26k トークン）でも global 層は 1 回 1.56 ms で，KV ヘッドが E2B の 4 倍あるのに時間はほぼ同じである．**時間を決めているのは読むバイト数ではない**ことを示している．vLLM は head dim が 256 と 512 の層が混在する Gemma 4 に対して全層で FlashAttention-4 を使っており，global 層の decode もこの経路を通る．理由はコード 7 のとおりで，Hopper 既定の FlashAttention 3 は head dim 256 までしか扱えないため，BF16 の KV キャッシュでは FA4 を全層に使う．FP8 の KV キャッシュを使う場合だけ，head dim 256 の local 層に FA3 を使う層ごとの選択に切り替わる（global 層は FA4 のまま．結果は後述）．

コード 7: vLLM が Gemma 4 の Attention カーネルを選ぶ処理（抜粋）．

```
# vllm/model_executor/models/config.py  class Gemma4Config (excerpt)
def verify_and_update_config(vllm_config):
    """Configure attention for heterogeneous head dimensions.

    Gemma4 uses different head dimensions for sliding window vs full attention
    layers. The default FA3 on Hopper cannot handle head_dim > 256.
    ...
    force FA4 for all layers to avoid the mixed FA3+FA4 penalty.
    """
    ...
    if is_fa_version_supported(4) and max_head_dim <= 512:      # <- global layers: 512
        ...
        if use_per_layer_fa:        # <- only on SM90 *with an FP8 KV cache*
            ...                     #    (FA3 for 256-dim layers, FA4 for 512-dim layers)
        else:
            vllm_config.attention_config.flash_attn_version = 4  # <- BF16 KV: FA4 for every layer
```

表 12 に，Attention カーネル 1 回の時間を KV の読み出し量と比べた．Gemma 4 の global 層は KV を理論帯域で読む時間の 28–117 倍かかり，実効帯域は 33–137 GB/s にすぎない．一方 Qwen3-32B（FlashAttention 3，head dim 128）は，26k トークンで 1 層あたり 106 MB の KV を 44 µs で読んでおり，実効帯域は 2.4 TB/s である（split-KV の部分結果をまとめるカーネル 5 µs を含めても 2.2 TB/s）．**読むバイト数が少ないのに遅いのは，モデルの設計ではなく，Gemma 4 の global 層に使われている Attention カーネルの実装に原因がある．**

■なぜ遅いか（Nsight Compute）． global 層のカーネルは理論どおりの 61.6 MB を読んでおり，余計な読み出しはない（表 13）．しかし DRAM も SM もピークの 1% 未満しか使っておらず，メモリ律速でも計算律速でもない．grid は 128 CTA で SM の数に近いので，一見すると SM は埋まっているように見える．ところが同じカーネルを単独で Nsight Compute にかけ，SM ごとの稼働サイクル（`sm__cycles_active`）を見ると，最大の SM は 274 万サイクルとカーネルの実行時間（305 万サイクル）のほぼ全体で動いているのに，平均は 3.6 万サイクル，最小は 0 である．**128 個の CTA のうち 1 個だけがすべての KV を読み，残りの 127 個は何もせずに終**

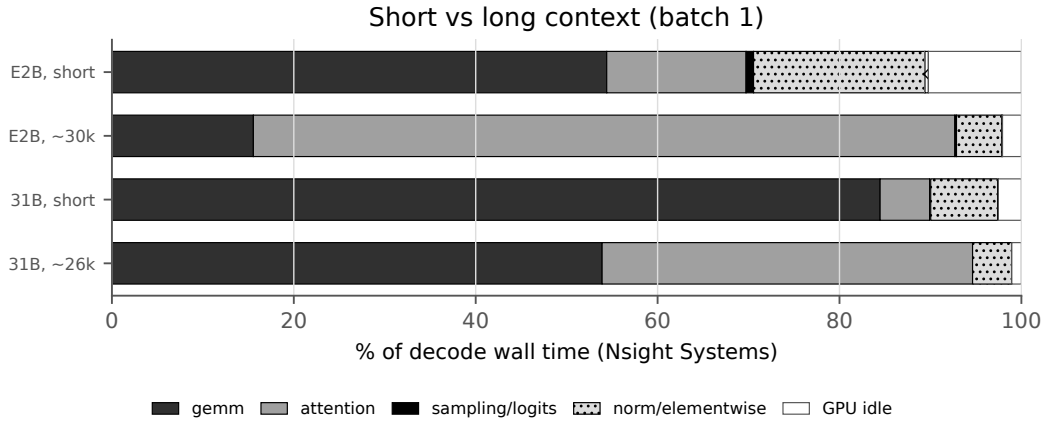


図 20: 短いコンテキストと長いコンテキストでの decode 1 ステップの時間内訳（Nsight Systems，バッチ 1）．

表 12: 長いコンテキストの decode における Attention カーネル 1 回の時間と KV の読み出し量．理論時間は KV / 3.9 TB/s．Gemma 4 は CuTe DSL の FA（FA4 系），Qwen3-32B は FlashAttention 3．修正後の時間は split の部分結果をまとめる combine カーネルを含む（E9）．

				vLLM 0.32.0		split-KV 修正後	
モデル	層	KV / 回	理論	実測	実効帯域	実測	実効帯域
Gemma4 E2B（30k）	global	61 MB	16 µs	1.84 ms	33 GB/s	44 µs	1.4 TB/s
Gemma4 E2B（30k）	local	0.5 MB	0.1 µs	23 µs	22 GB/s	15 µs	33 GB/s
Gemma4 31B（26k）	global	213 MB	55 µs	1.56 ms	137 GB/s	98 µs	2.2 TB/s
Gemma4 31B（26k）	local	17 MB	4 µs	38 µs	440 GB/s	23 µs	0.74 TB/s
Qwen3-32B（26k）	全層	106 MB	27 µs	44 µs	2.4 TB/s	–	–

わっている（図 5(b)）．実際，1.84 ms という時間は，修正後のカーネルを split 数 1（CTA 1 個）で動かしたときの時間（1.87 ms，図 21 右）とほぼ同じである．31B の global 層は KV ヘッドが 4 本で，仕事をする CTA も 4 個になるため，実効帯域は約 4 倍の 137 GB/s になる（表 12）．

■原因：split-KV の分割数が渡っていない． decode ではクエリが 1 トークンしかないので，FlashAttention は KV を系列方向に N 個に分け（split-KV），N 個の CTA が並列に部分結果を計算し，最後に combine カーネルでまとめる．vLLM 0.32.0 に同梱されている FA4（CuTe DSL）の Hopper 向け forward カーネル（`flash_fwd_sm90.py`）では，各 CTA が自分の担当範囲を計算する `BlockInfo` を作るときに分割数 `num_splits` を渡していない（既定値 1 になる）．範囲の計算（`get_n_block_min_max`）はこの値を使うので，split 0 には KV 全体が，split 1–127 には空の範囲が割り当てられる．combine カーネルは空の部分結果を無視するので**出力は正しく**，そのためにこのバグは表に出にくい．修正は `BlockInfo` の生成に `num_splits=num_splits` の 1 行を足すだけである．カーネル単体で測ると，修正前は split 数を 1 から 128 まで変えても時間が 1.87 ms で変わらないのに対し，修正後は split 数にほぼ反比例して縮み，128 split で 49 µs（38 倍速，1.26 TB/s）になる（図 21）．head dim 256 の FA3 に対しても，読むバイトあたりで同等以上の速さになる．

■Attention backend を替えると decode は速くなるが，prefill は遅くなる． バグを特定する前に，原因がこのカーネルにあることを確かめるため，同じモデルを Triton の Attention backend（`--attention-backend TRITON_ATTN`）で測り直した（E8，表 14）．30k での TPOT は E2B で 17.0 ms から 6.7 ms（2.5 倍速），31B で 40.7 ms から 27.5 ms（1.5 倍速）に縮んだ．一方で 32k の TTFT は E2B で 1.9 倍，31B で 1.65 倍に伸びる．prefill では FA4 の方が Triton より速いからである．FP8 の KV キャッシュでは local 層が FA3 に替わるが，31B の 30k での TPOT は 41.0 ms で変わらなかった．時間の大半を占める global 層が FA4 のままだから

表 13: Nsight Compute：E2B，約 30k トークン後の decode の FlashAttention-4 カーネル（1 ステップ分 35 回の平均）．CuTe DSL カーネルのためアプリケーションリプレイ・eager 実行・ベースクロック固定で計測しており，絶対時間は Nsight Systems より長い．

	local 層（28 回）	global 層（7 回）
グリッド / ブロック	128 CTA / 384 thr	128 CTA / 384 thr
実行時間（ベースクロック）	37 µs	2.78 ms
DRAM 読み出し	0.63 MB	61.6 MB（理論 61.4 MB）
DRAM スループット（ピーク比）	0.43%	0.58%
SM スループット（ピーク比）	9.0%	0.47%
アクティブ warp（ピーク比）	17%	18%

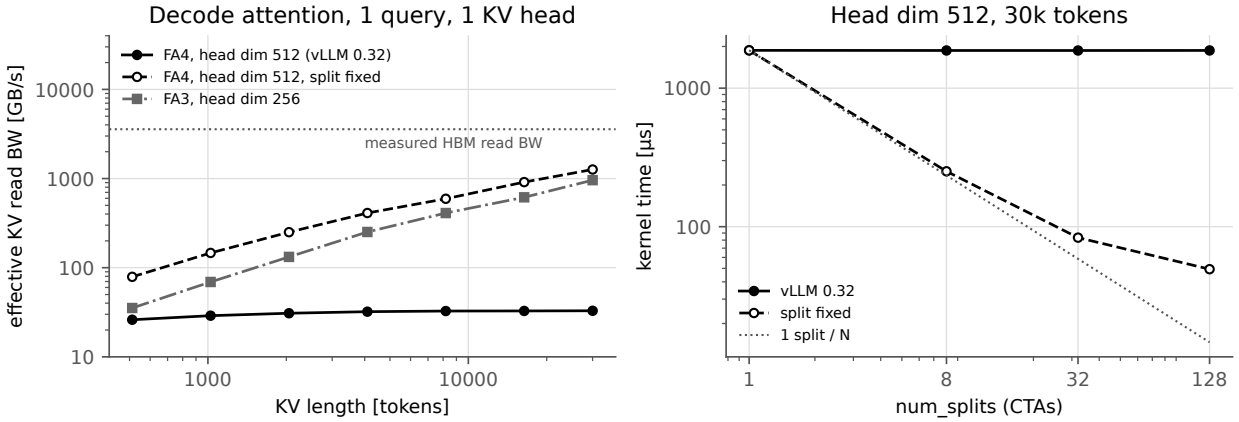


図 21: global 層（head dim 512）の decode Attention カーネル単体の測定（KV ヘッド 1 本，ページサイズ 16, E9）．左：KV 長と実効帯域．vLLM 0.32.0 の FA4 は KV 長によらず 1 SM 分の帯域（約 33 GB/s）で頭打ちになる．右：30k トークンでの split 数と時間．修正前は split 数を増やしても時間が変わらない．点線は 1 split の時間を split 数で割った理想．

である（E2B は FP8 KV キャッシュでは CUDA Graph の取り込み中にエラーとなり，起動しなかった）．

8.4 split-KV を修正した FA4 での測り直し

修正の効果をサービング全体で確かめるため，1 行の修正を入れた FA4 で E1 の単一ストリーム計測と E6 の長いコンテキストのプロファイルを測り直した（E9）．インストールした vLLM 自体は書き換えておらず，計測用のサーバプロセスの中でだけ，範囲を計算する関数を修正版に差し替えている．それ以外の条件（モデル，オプション，ワークロード）は元の計測と同じである．

- **長いコンテキストの decode の遅さはほぼ消えた．**30k での TPOT は E2B で 17.0 ms から 3.7 ms（4.6 倍速），31B で 40.7 ms から 23.3 ms（1.7 倍速）になった（表 11，14）．TPOT の増分は 0.1–1.3 ms で，KV 読み出しの理論増分の 0.8–1.8 倍に収まり，Qwen3 と同じ水準になった．
- **カーネル 1 回の時間は 16–42 倍短くなった．**global 層 1 回は E2B で 1.84 ms から 44 µs，31B で 1.57 ms から 98 µs になり（combine を含む），実効帯域は 1.4–2.2 TB/s に上がった（表 12）．E2B の 30k での decode 1 ステップに Attention が占める割合は 81% から 17% に下がった．
- **短いコンテキストの decode も 0.6–1.0 ms 速くなった．**local 層のカーネルも 128 split のうち 1 個しか働いていなかったからで，E2B の local 層 1 回は 26 µs から 15 µs になった．
- **prefill と精度は変わらない．**TTFT の差は 32k で +2–6% にとどまり，Babilong の正答数も同じだった（E2B のみ 36 問中 16 問から 15 問）．split の部分結果をまとめる順序が変わるため，BF16 の丸めの違い

で greedy 生成の出力が途中から分かれるリクエストはある．

- **Triton backend より decode も prefill も速い．**修正後の FA4 は 30k の TPOT で Triton を E2B で 1.8 倍，31B で 1.2 倍上回る．Triton で decode が速くなったのは Triton が速いからではなく，FA4 が壊れていたからである．
- **高並列のスループットは変わらない．**31B の同時実行スイープ（E2）でも，修正後の出力スループットは 32 並列で 507 tok/s，64 並列で 494 tok/s で，修正前（533，522 tok/s）と同程度だった．SWE-bench のプロンプトは 1.3k トークン程度と短く，global 層の Attention が時間に占める割合が小さいからで，§8.5 の頭打ちの原因はこのバグではない．

表 14: Attention backend と KV キャッシュの型による TPOT と TTFT の違い（ms，中央値）．TPOT（<1k）は 1k トークン未満のプロンプト，∼30k と TTFT は Babilong の各長さ設定（32k 設定は約 31k トークン）．

モデル	Attention backend	TPOT（<1k）	TPOT（∼30k）	TTFT（8k）	TTFT（32k）
Gemma4 E2B	FA4（既定）	4.20	16.96	105	602
Gemma4 E2B	Triton	3.51	6.73	150	1,122
Gemma4 E2B	FA4（split-KV 修正後）	3.59	3.70	108	613
Gemma4 31B	FA4（既定）	23.01	40.66	1,167	5,999
Gemma4 31B	Triton	21.85	27.53	1,529	9,915
Gemma4 31B	FA3/FA4（FP8 KV）	23.04	40.99	1,214	6,185
Gemma4 31B	FA4（split-KV 修正後）	22.02	23.27	1,225	6,309
Qwen3-32B	FA3（既定）	22.00	24.93	1,290	7,179

アーキテクチャ上は Gemma 4 の方が長いコンテキストの decode に有利（30k での KV 読み出しは同クラスの Qwen3 の 28–42%）であり，修正後はその利点がそのまま表れる．30k での TPOT は 31B が 23.3 ms で Qwen3-32B（24.9 ms）より 7% 速く，26B-A4B が 4.9 ms で Qwen3-30B-A3B（5.5 ms）より 12% 速い．vLLM 0.32.0 での「Gemma 4 は長いコンテキストの decode が遅い」という結果は，モデルではなくこのバグによるものだった．

コラム：upstream にコミットするチャンスは，すでに過ぎていた

このバグを見つけて 1 行の修正で 38 倍速くなったとき，筆者は「これは vLLM にプルリクエストを出せる」と思った．出力が正しいまま 128 個の CTA のうち 1 個しか働かないという，テストでは見つからない種類のバグであり，Gemma 4 の長いコンテキストの decode を 4.6 倍速くする．ところが最新の upstream を調べると，すでに直っていた．

- FA4 のカーネルは vLLM のリポジトリには含まれておらず，ビルド時に vllm-project/flash-attention の特定のコミットを取り込んでいる．
- そのリポジトリでは 2026 年 9 月 28 日に *Fix SM90 SplitKV block range partitioning*（#198）として，本書の修正と同じ 1 行の修正がマージされていた．
- vLLM 側では 10 月 13 日のコミット（#55866）で取り込むコミットが `9cd61de` から `b273a02` に更新され，v0.32.1rc0 以降（v0.33.0rc0/rc1 と main）には修正が入っている．このコミットには，head dim 512 の forward の shared memory の競合を直す修正（#203）も含まれている．
- 本書で使った v0.32.0 は 10 月 17 日のリリースで，main への修正より後に出ているが，リリース用のブランチには修正が入っておらず，修正前のコミットを取り込んでいる．執筆時点で修正を含むのは候補版（rc）だけで，正式リリースの最新版である v0.32.0 を使う利用者はこのバグの影響を受ける．

コミットのチャンスだと思ったのにすでに直っていて，正直なところ悔しい．それでも，このバグがハードウェアの上で何を起こしていたか（1 SM しか働かず，HBM 帯域の 1% で動いていた）と，修正でサービング性能がどれだけ変わるか（表 11）を定量的に示した報告は，筆者の調べた範囲では見つからなかった．v0.32.0 で Gemma 4 を長いコンテキストで動かしている場合は，修正を含む版（v0.32.1rc0 以降）に上げれば decode が最大 4.6 倍速くなる．

8.5 同時実行：バッチによる償却と，MoE・KV 容量の限界

同時実行数スイープ（E2）では，SWE-bench Lite のプロンプト（平均約 1.3k トークン）を出力 256 トークン固定で流した．

■バッチで重みの読み出しを償却できる． E2B のスループットは同時実行 1 の 189 tok/s から 64 の 6,869 tok/s（36 倍）まで伸び，TPOT は 4.4 ms から 7.7 ms にしか悪化しない（図 22 左）．重みはバッチ全体で 1 回しか読まないからである．64 並列でも dense の arithmetic intensity は 30–50 FLOP/byte で ridge point（138–214）に届かず，decode はまだ帯域律速の領域にある（同右）．MoE の 2 モデルは 64 並列でも 7–8 FLOP/byte にとどまる．バッチを大きくすると読む expert も増えるので，FLOPs と読み出しバイトが一緒に増え，比があまり上がらないためである（§8.5）．

■高負荷では prefill の計算が TPOT を押し上げる． 12B の TPOT は 64 並列で 31.6 ms で，decode だけの roofline 下限（11.8 ms）の 2.7 倍である（図 23）．chunked prefill では，定常状態で出力 1 トークンあたり約 5 トークン分の新規リクエストの prefill が同じステップに入る．その計算時間（実測 GEMM ピーク 492 TFLOPS で 18.0 ms，514 TFLOPS なら 17.2 ms）を，HBM の読み出し時間（11.8 ms）と重ならずに足し合わせた見積もり（破線，492 TFLOPS の場合）は 29.0–29.9 ms になり，同時実行数による TPOT の増え方をよく再現する．同時実行 1 でも残る約 4.5 ms の差は，§8.2 で見た固定的な latency である．入力が長く出力が短いワークロードほどこの影響は大きく，prefill/decode の分離や `max-num-batched-tokens` の調整が効く領域である．

■MoE の「アクティブ 3.8B」は単一ストリームでのみ成り立つ． 26B-A4B の TPOT は 5.7 ms から 23.7 ms へと急に悪化し，64 並列では dense の 12B に近づく（図 24）．式 (4) の $f(B)$ により，1 ステップで読む重みが $B = 1$ の 7.7 GB から $B = 64$ で 50 GB まで増え，12B（24 GB）の 2 倍になるからである（expert が等確率・独立に選ばれると仮定した理論値．偏りがあれば実際はこれより少ない）．Qwen3-30B-A3B も同じ挙動で，最大

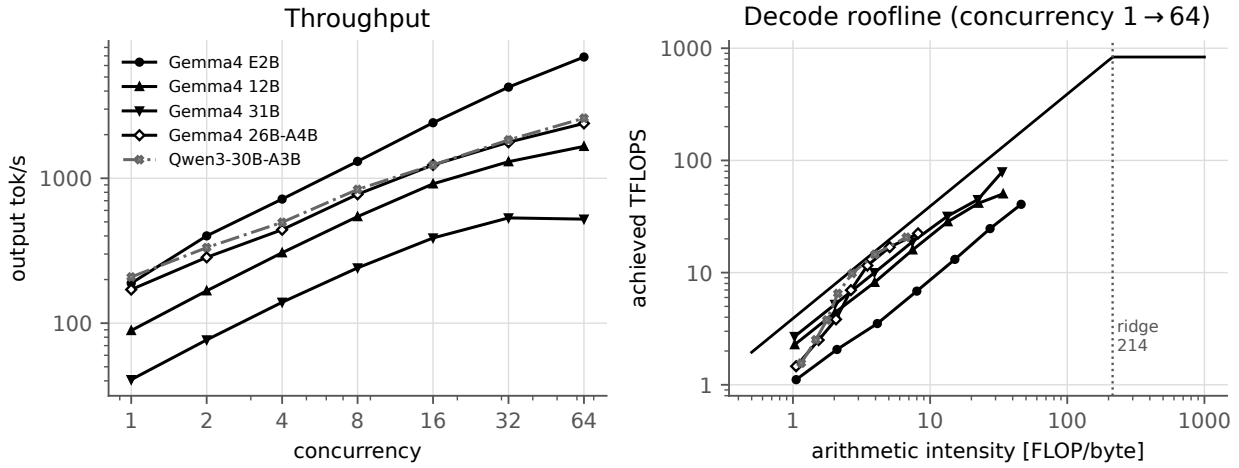


図 22: 同時実行数に対するスループット（左）と，decode ステップの roofline 上の位置（右，arithmetic intensity は理論 FLOPs / 理論バイト，到達性能は理論 FLOPs / 実測 TPOT）．dense 3 モデルと MoE 2 モデル．

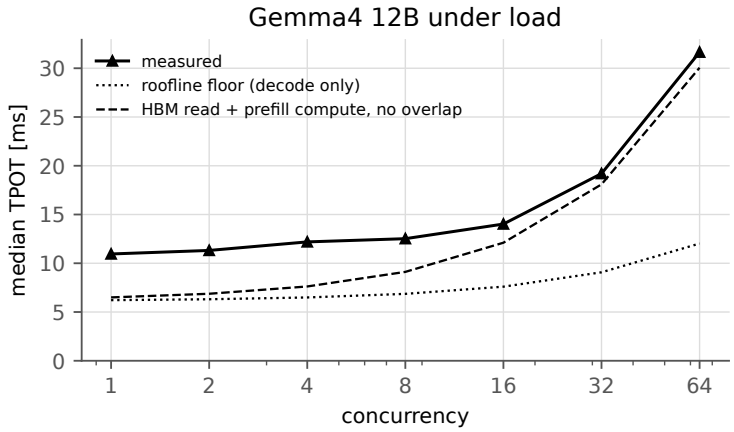


図 23: 12B の TPOT と同時実行数．点線は decode だけの roofline 下限，破線は HBM 読み出しと prefill の計算が重ならないとした見積もり．

スループットは 26B-A4B が 2,392 tok/s，Qwen3-30B-A3B が 2,599 tok/s とほぼ同じである．

■31B は 32 並列で頭打ちになり，KV が足りなくなる． 31B は 32 並列の 533 tok/s から伸びが止まり，64 並列では TTFT が 18 s に跳ね上がる（図 25）．サーバログでは KV キャッシュ使用率が 98.6%，同時に実行されたリクエストは最大 36 本で，残りは待ち行列に入っていた．重みを載せた後の KV 領域は 27 GiB しかない．さらに約 1.8k トークンのリクエストは window size（1024）を超えて local 層が満杯になるため，1 トークンあたり平均の KV が Qwen3-32B の 2.1 倍（535 対 256 KiB）になる（図 12）．その結果，64 並列では Qwen3-32B（839 tok/s）が 31B を 1.6 倍上回った．

■FP8 KV キャッシュで容量を倍にしても，スループットは伸びない． KV 容量がスループットを決めているのかを確かめるため，31B を FP8 の KV キャッシュで同じ条件で測った（E8）．KV 領域は 59,026 トークンから 114,656 トークン（1.94 倍）に増え，64 並列でも 64 本すべてが同時に走るようになった．その結果 64 並列の TTFT は 18.2 s から 4.4 s に縮んだが，TPOT は 52 ms から 91 ms に伸び，出力スループットは 522 tok/s から 527 tok/s とほとんど変わらなかった．同時に走るリクエストが最大 36 本から 64 本に増えた分だけ 1 ステップが長くなり，相殺されたのである．8–32 並列では FP8 KV の方がむしろ 5–9% 低い（32 並列で 487 対 533 tok/s）．KV を FP8 に変換する処理や層ごとの backend 切り替えの上乗せと考えられるが，本書では分解していない．入

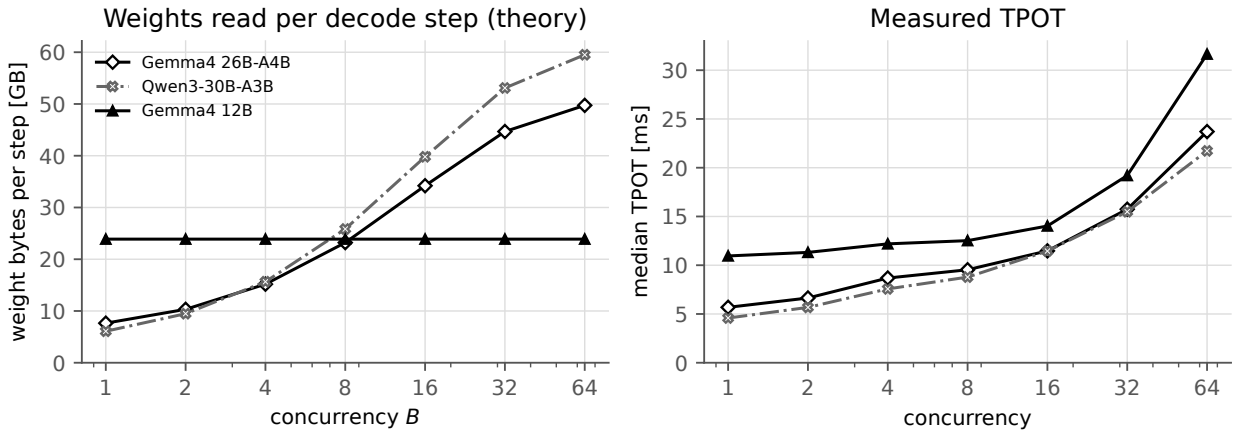


図 24: MoE 2 モデルと dense の 12B について，1 ステップで読む重みの理論値（左，expert が等確率・独立に選ばれると仮定）と実測 TPOT（右）．

力を含めた処理トークン数は，64 並列で 31B が約 3,000 tok/s，Qwen3-32B が約 4,300 tok/s である．パラメータ数はほぼ同じなので，31B は 1 トークンを処理する効率そのものが 3 割低い．したがって **31B の高並列スループットの頭打ちは KV 容量ではなく，1 ステップの処理の効率で決まっている**．KV の不足は待ち行列を生んで TTFT を悪化させるが，スループットの上限は決めていない．§8.3 の split-KV のバグを修正してもスループットは変わらなかったので（§8.4），このバグは原因ではない．どのカーネルが効率を下げているかは，本書では分解していない．

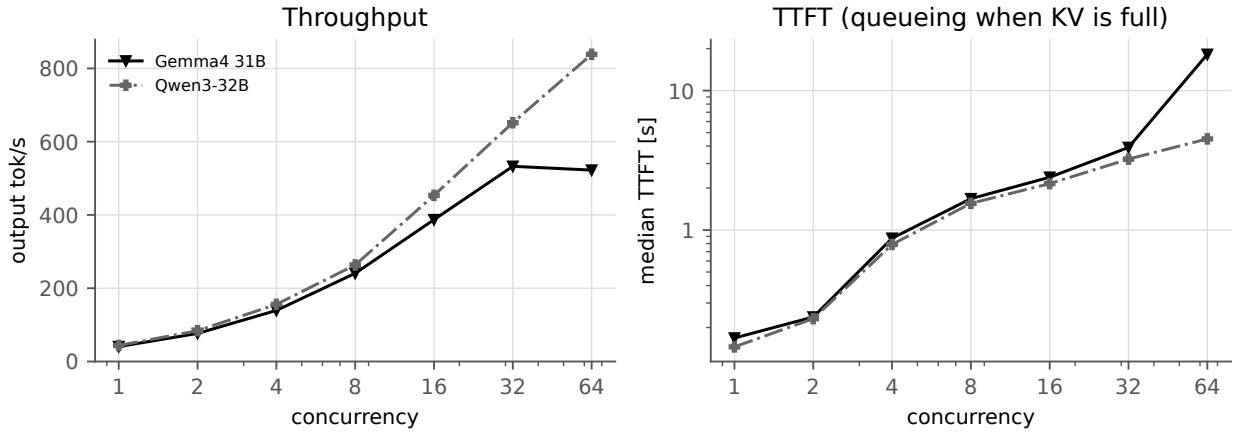


図 25: 31B と Qwen3-32B のスループット（左）と TTFT（右）．31B は 32 並列を超えると KV が足りず，リクエストが待ち行列に入る．

■vLLM の KV 割り当て（E4）． Qwen3 では vLLM が見積もる最大同時実行数が理論値と一致するが，Gemma 4 の 12B/26B/31B では理論値の 1/4–1/6 しか確保されていない（表 15）．31B で最大長 32,768 の起動に必要と報告された 27.5 GiB は，全層をフルコンテキストとした場合の理論値（27.50 GiB）と一致しており，少なくとも起動時のメモリ見積もりは sliding window の分を差し引いていない．運用中の割り当てでも，local 層（head dim 256，KV ヘッド 8–16）と global 層（head dim 512，KV ヘッド 1–4）でページの大きさを揃えるためのパディングが理論値との差を生んでいると考えられるが，本書では確認していない．

表 15: vLLM が確保する KV 領域と，最大長リクエストの最大同時実行数．ideal は窓と KV 共有を考慮した理論 KV 量（K と V を両方格納）で割った値，no window は全層をフルコンテキストとした場合．

Model	KV pool [GiB]	req. len	ideal KV/req [GiB]	max conc. (vLLM)	max conc. (ideal)	max conc. (no window)
Gemma4 E2B	74.2	40k	0.24	173.2	309	105.5
Gemma4 E4B	68.8	40k	0.64	54.2	107	31.5
Gemma4 12B	61.0	40k	0.94	10.3	65	4.6
Gemma4 26B-A4B	35.1	40k	0.98	8.6	36	4.1
Gemma4 31B	27.0	34k	3.44	1.7	8	0.9
Qwen3-30B-A3B	27.1	40k	3.75	7.2	7	7.2
Qwen3-32B	24.8	40k	10.00	2.5	2	2.5

表 16: speculative decoding の方式・draft length 別の mean acceptance length AL と TPOT 高速化（中央値）．n-gram はプロンプト内の n-gram 照合（モデル不要）．Babilong* は出力長を固定するため EOS を抑制しており，回答後に同じ文を繰り返す（acceptance rate が高く出る）ので参考値．

Model	method	Synthetic AL	Synthetic speedup	SWE-bench AL	SWE-bench speedup	Babilong* AL	Babilong* speedup
Gemma4 E2B	MTP γ=1	1.63	1.46×	1.81	1.66×	1.66	1.47×
Gemma4 E2B	MTP γ=2	1.96	1.59×	2.42	2.05×	2.07	1.62×
Gemma4 E2B	MTP γ=4	2.28	1.58×	3.22	2.37×	2.67	1.75×
Gemma4 E2B	MTP γ=8	2.46	1.33×	4.10	2.28×	3.08	1.48×
Gemma4 E4B	MTP γ=4	2.43	1.78×	3.32	2.66×	2.19	1.60×
Gemma4 12B	MTP γ=4	2.71	1.63×	3.63	1.98×	3.41	2.15×
Gemma4 26B-A4B	MTP γ=1	1.83	1.15×	1.85	1.03×	1.80	1.24×
Gemma4 26B-A4B	MTP γ=2	2.45	1.28×	2.55	1.16×	2.42	1.40×
Gemma4 26B-A4B	MTP γ=4	3.31	1.43×	3.59	1.30×	3.24	1.37×
Gemma4 26B-A4B	MTP γ=8	4.01	1.20×	4.58	1.15×	4.08	1.18×
Gemma4 31B	MTP γ=1	1.86	1.77×	1.86	1.79×	1.89	1.80×
Gemma4 31B	MTP γ=2	2.49	2.27×	2.57	2.37×	2.68	2.50×
Gemma4 31B	MTP γ=4	3.54	3.01×	3.61	3.14×	3.98	3.55×
Gemma4 31B	MTP γ=8	4.27	3.14×	4.55	3.59×	5.86	4.79×
Gemma4 31B	n-gram γ=4	1.03	0.95×	1.42	1.32×	3.31	3.32×

8.6 MTP Drafter による speculative decoding

■AL は似ているのに，高速化は大きく違う． $\gamma = 4$ の TPOT 高速化は 31B 3.08 倍，E4B 2.17 倍，E2B 1.94 倍，12B 1.85 倍，26B-A4B 1.35 倍である（図 26，Synthetic + SWE-bench）．31B は vLLM PR の報告（$\gamma = 8$ で 3.19 倍）とよく一致する．一方 AL は 2.8–3.6 トークンとモデル間の差が小さく，26B-A4B は 2 番目に高い．つまり 26B-A4B が効きにくいのは drafter の予測が外れるからではなく，1 ステップのコスト c が大きいからである（後述）．

■生成内容への依存． コードのパッチ生成（SWE-bench）は自然文（Synthetic）より AL が高い（E2B で 3.22 対 2.28）．差分の文脈行はコードのコピーで予測しやすいためである．リポジトリ別では sympy が最も低く，matplotlib が最も高い（付録の図 32）．モデル不要の n-gram（31B，$\gamma = 4$）は，プロンプトからのコピーが多い SWE-bench で 1.32 倍，繰り返しの多い Babilong* で 3.32 倍だが，自由な文章では 0.95 倍とかえって遅くなる（図 27）．MTP はどのワークロードでも 3.0–3.5 倍を出しており，**学習済み drafter の強みは内容によらない acceptance rate にある**．

■なぜ 26B-A4B では効きにくいか：コストモデルによる説明． 式 (3) の AL と c を分けると（図 30），26B-A4B の AL（3.46）は 31B（3.58）とほぼ同じで，差は c にある．26B-A4B は $\gamma = 4$ の verify で 5 トークン分の expert（一様・独立の仮定で 27.5%．$B = 1$ では 6.3%）を読むので，verify だけで通常ステップの 2.3 倍のバイトを読む．drafter（target の 11%）4 回分を足すと式 (7) で $c_{\text{pred}} = 2.78$ となり，実測の $c = 2.56$ と近い．一方 31B は dense なので verify がほぼ無料で，$c_{\text{pred}} = 1.12$（実測 1.16），予測高速化 3.20 倍（実測 3.08 倍）とよく一致する．

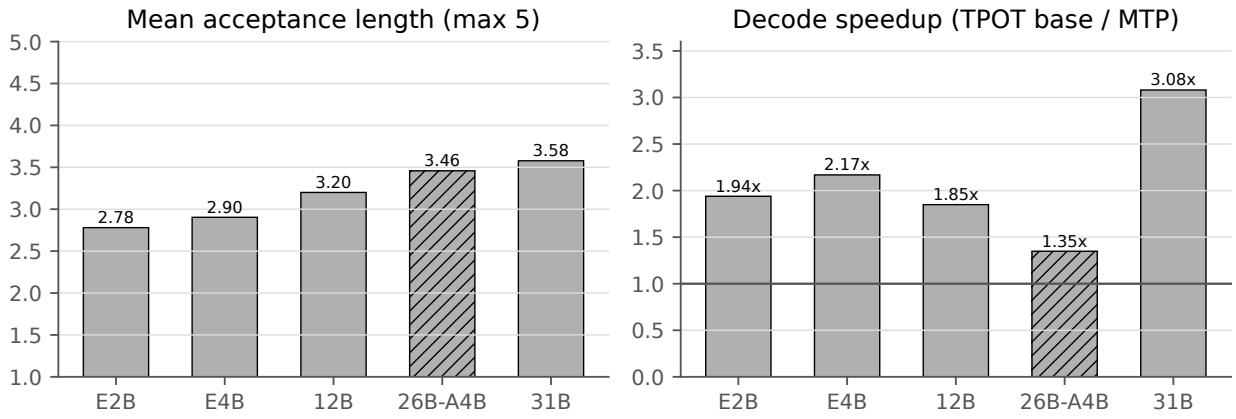


図 26: MTP（$\gamma = 4$，同時実行 1，Synthetic + SWE-bench）の mean acceptance length（左）と TPOT 高速化（右）．斜線は MoE の 26B-A4B.

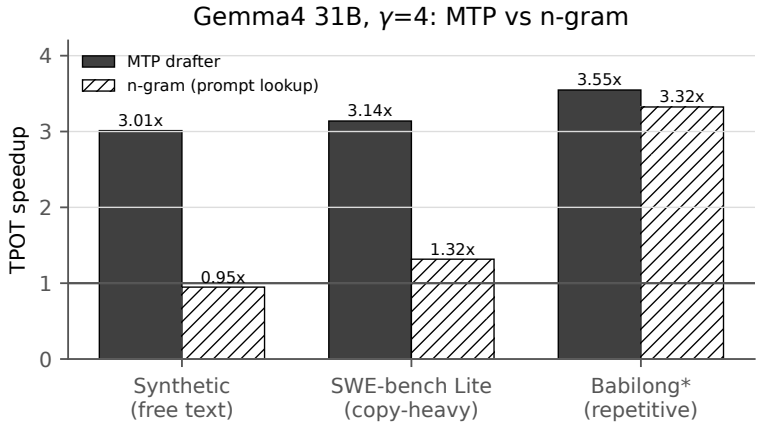


図 27: 31B（$\gamma = 4$）での MTP と n-gram の TPOT 高速化．Babilong* は回答後に同じ文を繰り返すので参考値.

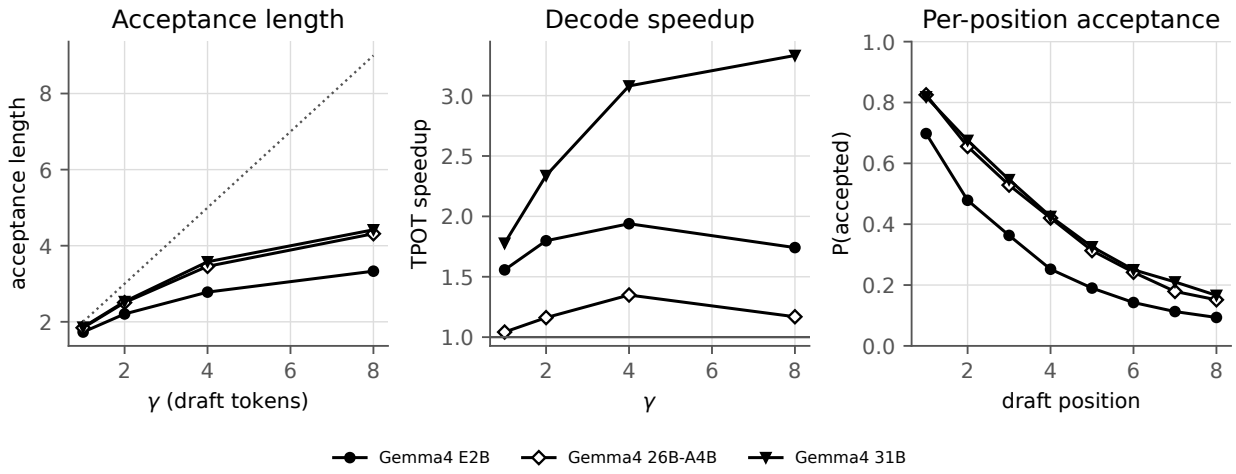


図 28: draft length γ の影響（Synthetic + SWE-bench）．左：AL（点線は全 accept の上限 $\gamma + 1$），中：TPOT 高速化，右：$\gamma = 8$ での位置ごとの acceptance rate.

E2B/E4B/12B は実測の c が予測より 0.3–0.5 大きい．E2B/E4B は 1 ステップが 2–4 ms と短く，drafter 1 回分の固定 latency が相対的に重い．そのため E2B は $\gamma = 8$ で $\gamma = 4$ より遅くなる．12B と 26B-A4B では MTP 有効時に GPU が約 50% の時間 idle になっていた（図 29）．26B-A4B は別の日に測り直しても，高速化が $\gamma = 4$ で 1.41 倍，$\gamma = 8$ で 1.24 倍，GPU busy が 48%（通常の decode は 92%，31B の MTP は 90%）で，同じ結果になった（E8）．drafter の経路で 1 出力トークンあたり CUDA Graph 起動が 12–20 回，通常のカーネル起動が 86–114 回発生しており（通常の decode ではそれぞれ 1.1 回と 25 回），CPU が律速になっている．drafter の経路を完全に CUDA Graph 化できれば，12B の高速化は 1.85 倍から予測値の 2.7 倍に近づく．**最適な γ は，大きい dense モデルで 8 以上，小さいモデルで 4 前後，MoE で 4（低負荷時）である．**

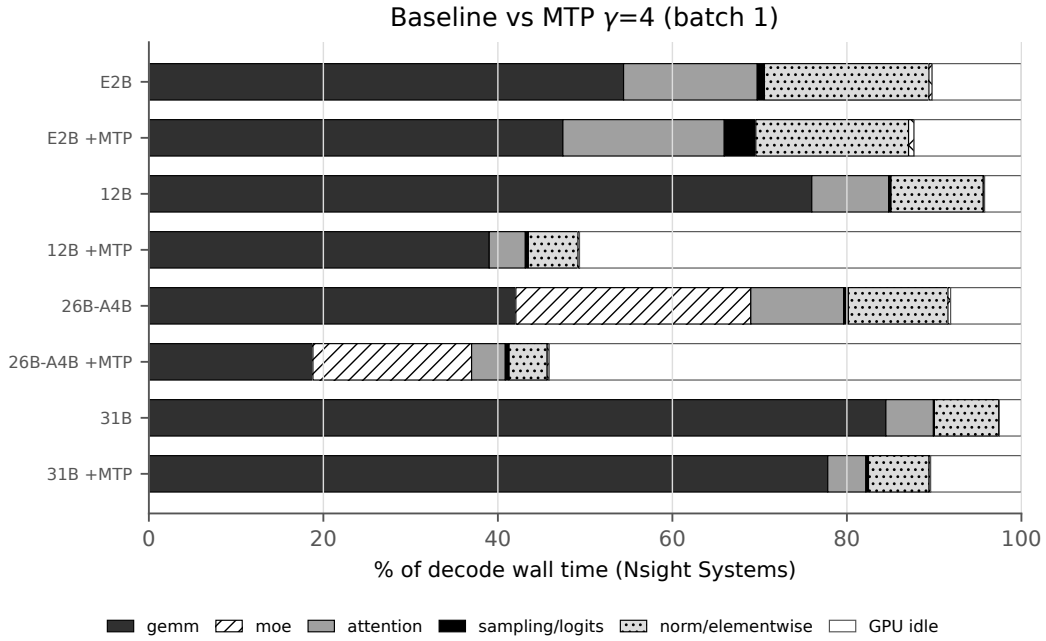


図 29: 通常の decode と MTP（$\gamma = 4$）の時間内訳（Nsight Systems，バッチ 1）．12B と 26B-A4B では MTP 有効時に GPU の idle（白）が約半分を占める．

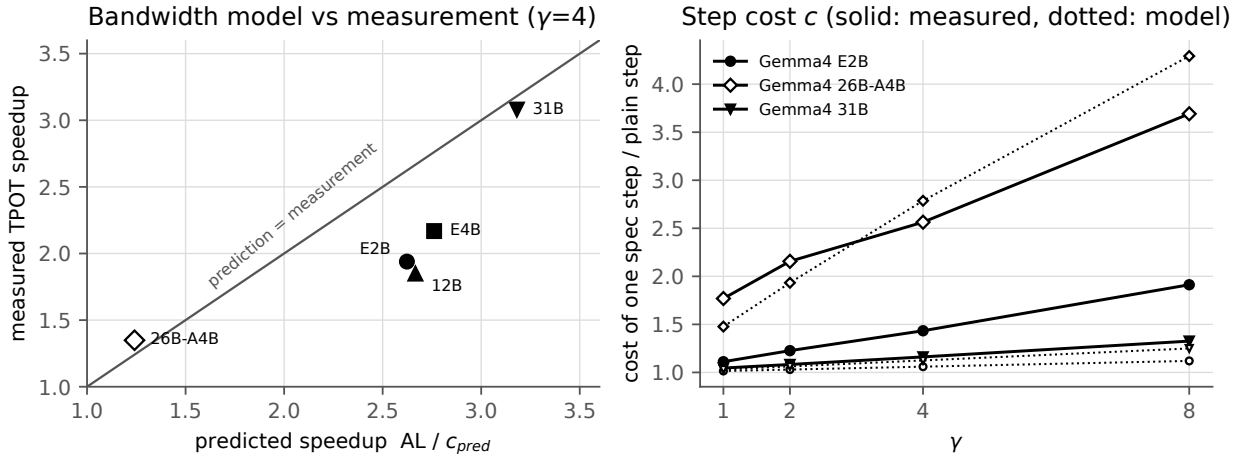


図 30: 帯域コストモデルによる予測高速化 $\mathrm{AL}/c_{\mathrm{pred}}$ と実測（左，$\gamma = 4$）．γ を振った 3 モデルの 1 speculative step のコスト c（右，実線：実測 AL/高速化，点線：式 (7)）．

■高負荷時の MTP． 同時実行数を上げると dense モデルの MTP の利得は縮む（図 31）．E2B では同時実行 1 の 2.3 倍が 64 では 1.16 倍になる．バッチが大きくなると arithmetic intensity が上がり，verify トークン分の計算が無料ではなくなるためである．逆に 26B-A4B は 1.25 倍から 1.51 倍へと利得が**増える**．高負荷ではもともと expert の大半を読んでいるので，verify トークンを増やしても読み出しバイトがほとんど増えないからである．vLLM PR が報告した「26B-A4B では MTP の利得がバッチとともに増える」傾向と一致する．

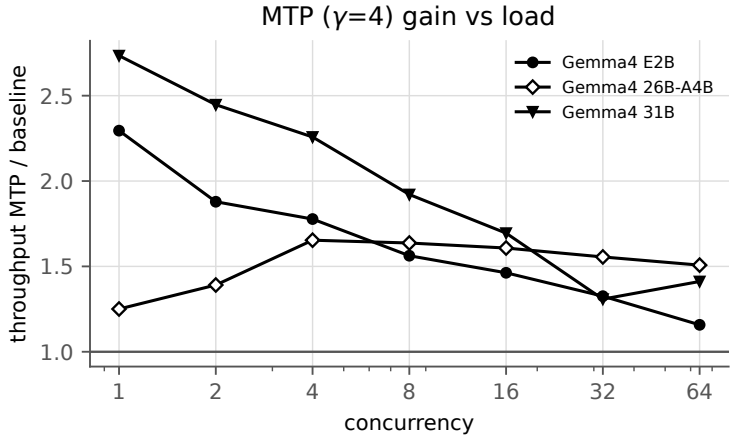


図 31: 同時実行数ごとの MTP（$\gamma = 4$）のスループット向上率（条件は図 22 と同じ）．dense の E2B・31B と MoE の 26B-A4B で傾向が逆になる．

9 外部の結果との比較と総合評価

9.1 外部の結果と整合しているか

本書の数値を，vLLM の Gemma 4 MTP 実装 PR[6]，第三者による H100 でのベンチマーク [12, 13]，Google の MTP 紹介記事 [5] と比べた（表 17）．外部の測定は GPU（H100 SXM），vLLM のバージョン，Attention backend，ワークロードが本書と異なるので，比べられるのは傾向と桁である．

表 17: 外部の結果との比較（同時実行 1 は出力 tok/s，括弧内は baseline に対する倍率）．外部の測定はいずれも H100 SXM（HBM 3.35 TB/s）と SPEED-Bench を使っており，jarvislabs は Attention backend に Triton を使っている．

指標	本書（H100 NVL，vLLM 0.32）	vLLM PR[6]	jarvislabs[12]	整合
31B baseline，同時実行 1	42 tok/s（TPOT 23.6 ms）	42.6 tok/s	40.3 tok/s（TPOT 24.4 ms）	○
31B MTP $\gamma = 8$，同時実行 1	148 tok/s（3.33×）	136 tok/s（3.19×）	125 tok/s（3.11×）	○
31B baseline，同時実行 16	387 tok/s（TPOT 32.2 ms）	–	375 tok/s（TPOT 32.9 ms）	○
26B-A4B baseline，同時実行 1	175 tok/s	169 tok/s	177 tok/s	○
26B-A4B MTP，同時実行 1	226 tok/s（1.35×，$\gamma = 4$）	270 tok/s（1.60×，$\gamma = 4$）	264 tok/s（1.49×，$\gamma = 8$）	△ 本書が低い（再計測でも 1.41×）
26B-A4B の MTP 利得と負荷	1.25→1.51×（同時実行 1→64）	1.60→2.27×（バッチ 1→64）	–	○ 同じ傾向
E4B MTP $\gamma = 4$，同時実行 1	2.17×	1.78×	–	△ 本書が高い

- **31B の baseline と MTP はよく一致する．**baseline の出力速度は 3 つの測定で 40–43 tok/s，MTP の高速化は 3.1–3.3 倍にそろう．同時実行 16 での TPOT も 32–33 ms で一致する．H100 NVL の HBM 帯域は SXM より 16% 大きいが，31B の baseline はほぼ同じである．重みの読み出しから求めた実効帯域はどちらも約 2.6 TB/s で，理論帯域に対しては本書が 67%，SXM が 78% になる．帯域の大きい NVL でも，実際に引き出せる帯域の絶対値は SXM と変わらないことを示唆している．
- **26B-A4B の MoE で MTP が効きにくいことも共通している．**Google の記事も 26B の MoE はバッチ 1 で routing が難しく，バッチを増やすと利得が増えると述べており [5]，vLLM PR の表も同じ傾向

を示す．これは §8.6 のコストモデル（verify で読む expert が増える）と整合する．ただし本書の高速化（1.35 倍）は外部（1.49–1.60 倍）より低い．baseline はほぼ同じなので，差は MTP 側にある．測り直しても 1.41 倍で外部より低く，baseline の TPOT も ±2% で一致したので，測定のばらつきではない．本書では 26B-A4B の MTP 有効時に GPU が約半分 idle になっており（図 29，再計測でも GPU busy 48%），CPU 側の処理が律速していた．CPU の性能や vLLM のバージョンで CUDA Graph 化される範囲が変わるとこの部分が大きく変わるため，それが差の原因と考えられる．

- **高負荷のスループットはワークロードの形で大きく変わる．**inferencebench は H100 SXM の 31B で同時実行 32 のとき 855 tok/s（入力 2048，出力 1024 トークン）を報告しており [13]，本書の 533 tok/s（入力約 1.3k，出力 256 トークン）より高い．出力 1 トークンあたりの prefill 量は先方が 2 トークン，本書が 5.2 トークンであり，§8 の prefill 混入の議論から，出力が長いほどスループットが高くなるのは整合する．
- **長いコンテキストの decode の遅さを定量的に報告した外部の結果は見つからなかった．**原因の split-KV のバグは flash-attention 側で 2026 年 9 月に修正済みで（§8.3 のコラム），vLLM v0.32.1rc0 以降には入っている．また head dim 512 の global 層が Attention backend の選択を難しくしていることはコミュニティでも知られている．vLLM には，head dim 混在のため全層で Triton に切り替わり E4B が RTX 4090 で約 9 tok/s しか出ないという報告 [14] や，local 層だけ FlashAttention を使えるようにする PR[15]（2026 年 10 月時点で未マージ）がある．jarvislabs が Triton backend で測っているのに対し，本書の vLLM 0.32 は FA4 を選ぶ（コード 7）など，バージョンによって使われるカーネル自体が変わっている．本書でも backend を Triton に替えると 30k での TPOT が 1.5–2.5 倍速くなり，prefill が 1.7–1.9 倍遅くなった（表 14）．

9.2 Qwen3 と比べて良い結果か

結論は，**アーキテクチャとしては Gemma 4 が優れている点が多いが，現在の vLLM の上ではその一部しか性能に表れておらず，条件によっては Qwen3 に負ける**である．

表 18: 同クラスの Qwen3 に対する Gemma 4 の評価．○：Gemma 4 が良い，△：同等，×：Gemma 4 が悪い．

観点	評価	根拠	原因
長いプロンプトの TTFT	○	32k で 16–33% 短い	Hybrid Attention（設計）
単一ストリームの decode（短いコンテキスト）	△	31B 22.9 対 22.0 ms，26B-A4B 5.3 対 4.6 ms	ほぼパラメータ数どおり
MTP を使った単一ストリームの decode	○	31B は 3.1–3.3 倍で Qwen3-32B（baseline）の約 3 倍速い	公式 drafter（設計）
長いコンテキストの decode	× 修正後 ○	v0.32.0 では 30k で TPOT の増分が 9–18 ms（Qwen3 は 1–3 ms）．split-KV の修正後は 31B 23.3 対 24.9 ms，26B-A4B 4.9 対 5.5 ms	FA4 の split-KV のバグ（実装，v0.32.1rc0 で修正）
長いコンテキストの KV 量（理論）	○	32k で 31B 3.3 GiB 対 8.0 GiB	Hybrid Attention（設計）
短いリクエストの高並列スループット	×	64 並列で 31B は Qwen3-32B の 0.6 倍．FP8 KV で KV 容量を倍にしても変わらない	1 トークンの処理効率（実装と考えられる）
MoE の高負荷スループット	△	2,392 対 2,599 tok/s	両者とも expert の読み出しが増える

表 18 で × がついた 2 点のうち，長いコンテキストの decode は完全に実装の問題であり（1 行の修正で消え，修正後は Qwen3 より速い），高並列のスループットも KV 容量ではなく 1 トークンの処理効率で決まっていた．つまり Gemma 4 の「残念な」結果の多くは**ソフトウェアで直せる**種類のものである．逆に，○の項目のうち MTP

による高速化は vLLM ですでに使え，外部の測定でも再現している．利用者の立場では，現時点で次のように使い分けるのがよい．

- **低負荷・低 latency の対話**：31B + MTP（$\gamma = 4$–8）が最も速い．同クラスの Qwen3-32B の約 3 倍の速さで出力できる．
- **長いプロンプトを読ませる用途**（RAG，要約，長い文脈での対話）：prefill が速く，split-KV の修正後は decode も Qwen3 より速い Gemma 4 が有利である．ただし vLLM v0.32.0 では長いコンテキストの decode が 1.7–4.6 倍遅いので，修正を含む v0.32.1rc0 以降を使う．v0.32.0 のまま Attention backend を Triton にする回避策は，decode は速くなるが prefill が 1.7–1.9 倍遅くなり，修正版の FA4 にどちらでも劣る．
- **短いリクエストを大量にさばく用途**：31B のスループットは 64 並列で Qwen3-32B の 0.6 倍で，FP8 KV キャッシュでも伸びない．FP8 KV キャッシュは待ち行列を消して TTFT を 18 s から 4.4 s に縮めるので，TTFT を抑えたい場合には有効である．スループットが重要なら Qwen3-32B が有利である．

9.3 コミュニティとして何をすべきか

本書の測定から，効果が大きいと見積もれる順に挙げる．

1. **split-KV の修正を含む版をリリースし，利用者に知らせる．**vLLM v0.32.0 の FA4 は split-KV の分割数が渡っておらず，128 個の CTA のうち 1 個しか働かない（§8.3）．修正は flash-attention 側にも vLLM の main にもすでに入っているが，執筆時点の正式リリースは修正前の v0.32.0 である．修正すると 30k での TPOT は E2B で 17 ms から 3.7 ms，31B で 41 ms から 23 ms になる（§8.4）．出力が正しいまま遅くなる種類のバグはテストで見つからないので，「split 数を増やすと時間が縮む」ことを確かめる性能テストがあれば再発を防げる．Triton backend への切り替えや，local 層だけ FA3 を使う層ごとの選択（PR[15]）は，修正版の FA4 があれば不要になる．
2. **KV キャッシュ管理の改善．**起動時の見積もりが sliding window を差し引いていないこと，運用時の割り当てが理論値の 1/4–1/6 にとどまることを直せば，同じメモリでより多くのリクエストを同時に走らせられる．ただし FP8 KV キャッシュで容量を倍にした実験では，待ち行列が消えて TTFT は縮んだがスループットは変わらなかった（§8.5）．効果は主に TTFT に表れ，スループットを上げるには 1 ステップの処理効率の改善と組み合わせる必要がある．さらに K=V の global 層で RoPE 前の K を 1 本だけ保存すれば，global 層の KV を半分にできる．
3. **drafter 経路の CUDA Graph 化．**12B と 26B-A4B では MTP 有効時に GPU が半分 idle で（26B-A4B は再計測でも GPU busy 48%），CPU からのカーネル起動が律速している．完全に CUDA Graph 化できれば 12B の MTP は 1.85 倍から帯域モデルの予測（2.7 倍）に近づき，26B-A4B も外部の 1.5–1.6 倍に近づくと考えられる．負荷に応じて γ を自動で変える仕組みも，dense と MoE で最適値が逆向きに動く（§8.6）ことから有用である．
4. **ベンチマークの報告のしかた．**Gemma 4 は vLLM のバージョンによって Attention backend が Triton，FA4 などと変わり，ワークロードの入出力長でスループットも大きく変わる．結果を共有するときは，vLLM のバージョン，Attention backend，入力長・出力長，同時実行数を併記することが，比較できる知見を積み上げるうえで重要である．

10　ボトルネックとチューニングの指針

表 19: ボトルネックと対策のまとめ.

対象	律速要因	根拠	有効な対策
小モデル（E2B/E4B）の decode	カーネルの固定 latency	帯域利用率 28–39%，1 トークン約 600 カーネル	同時実行数を上げる（64 並列で 36 倍），MTP（1.9–2.2 倍），normalization・residual のカーネル融合
大 dense（31B）の decode	HBM 帯域	帯域利用率 70%，GEMM は実測ピークの 94%	重みの量子化（FP8/INT4），MTP $\gamma \geq 4$（3.1–3.3 倍）
MoE（26B-A4B）	負荷により変化	高負荷では読む expert が 25B 相当に増える	低負荷は MTP の効果が小さい．高負荷では MTP が効く（1.5 倍）
長いコンテキストの decode	FA4 の split-KV のバグ（v0.32.0）	理論値の 26–120 倍，128 CTA のうち 1 個だけが働く	修正を含む v0.32.1rc0 以降を使う（30k で 1.7–4.6 倍速，TTFT は不変）
高並列の TPOT	混入する prefill の計算	12B の 64 並列で 31.6 ms，HBM 読み出し＋ prefill 計算の見積もり 29.9 ms	prefill/decode の分離，`max-num-batched-tokens` の調整
KV 容量（高並列）	local 層の KV と vLLM の割り当て	31B は 1.8k トークンのリクエストで平均 535 KiB/トークン（Qwen3-32B の 2.1 倍），割り当ては理論値の 1/4–1/6．不足すると待ち行列で TTFT が 18 s に	FP8 KV キャッシュ（TTFT 18→4.4 s，スループットは不変），FP8 重み，RoPE 前の K を 1 つだけ格納する K=V の活用
Prefill	電力制限下の GEMM 性能	実測 GEMM ピークの 74–84%	FP8 GEMM（理論 2 倍）

11　まとめ

H100 NVL 1 基・vLLM 0.32 で Gemma 4 ファミリのサービング性能を測り，チェックポイントから求めた理論限界と比べた．4 つの問いへの答えを，結果とその理由の組でまとめる．

■Q1：GPU の性能をどこまで出し切れているか．

- 31B の decode は HBM 理論帯域の 70% に達した．これは GEMM が実測帯域ピークの 94% で重みを読めており，重みの読み出し以外の上乗せ（1 ステップ約 4.6 ms）が理想時間 17.2 ms に対して小さいからである．
- E2B の decode は 28% にとどまった．これは重みの読み出し以外に，小さな GEMM・normalization などのカーネル群・Attention・GPU idle で 1 ステップ約 2.7 ms かかり，重みを読む理想時間（1.3 ms）の 2 倍に達するからである．この上乗せはモデルの大きさによらずほぼ一定なので，読むバイトが少ないモデルほど不利になる．
- MoE の 26B-A4B は 38% だった．これは expert 1 つあたりの行列が小さく，expert の GEMM が帯域を使い切れないからである．
- prefill は実測 GEMM ピークの 74–84% に達した．データシート値に対して 45–49% にとどまるのは，H100 NVL が 400 W の電力制限で大きな GEMM のクロックを下げるからである．同じ理由で GEMM ピーク自体も計測ごとに 492–514 TFLOPS と揺れた．

■Q2：アーキテクチャ上の工夫は効いているか．

- 32k トークンの TTFT は Qwen3 より 16–33% 短かった．これは Hybrid Attention によって P^2 で増える Attention 計算が global 層だけに限られ，Attention が FLOPs に占める割合が 36% から 18% に下がるからである．window size 以下では差がなく，効果は Attention の割合が大きくなる 4k トークン以上で現れる．
- 一方，vLLM 0.32.0 では，コンテキストを 30k トークンまで伸ばしたときの TPOT の増分が 9–18 ms で，Qwen3 の 1–3 ms を大きく上回った（31B は 41 ms 対 Qwen3-32B の 25 ms）．これは FA4 カーネルの split-KV に分割数が渡っておらず，128 個の CTA のうち 1 個だけがすべての KV を読んでいたからである．global 層の実効帯域は 33–137 GB/s で，1 SM 分の帯域しか出ていなかった．1 行の修正で増分は 0.1–1.3 ms と理論値並みになり，30k での TPOT は 31B が 23.3 ms，26B-A4B が 4.9 ms で，同クラスの Qwen3 よりそれぞれ 7%，12% 速くなった．この修正はすでに upstream に入っており，v0.32.1rc0 以降で使える．
- 64 並列では 31B のスループットが Qwen3-32B の 0.6 倍にとどまり，TTFT は 18 s に達した．TTFT が長いのは，window size を超える約 1.8k トークンのリクエストでは local 層の KV が満杯になって 1 トークンあたり平均の KV が Qwen3-32B の 2.1 倍になり，さらに vLLM の KV 割り当てが理論値の 1/4–1/6 にとどまるため，27 GiB の KV 領域が尽きてリクエストが待たされるからである．一方スループットが低いのは KV 容量のためではない．FP8 KV キャッシュで KV 領域を 1.94 倍にすると TTFT は 4.4 s に縮んだが，スループットは 522 から 527 tok/s で変わらなかったからである．
- 26B-A4B の TPOT は同時実行 1 から 64 で 4 倍に悪化した．これはバッチ内のトークンが別々の expert を選ぶため，1 ステップで読む重みが 7.7 GB から 50 GB に増えるからである．

■Q3：MTP Drafter はどの条件でどれだけ速くなるか．

- 単一ストリームでは 31B が 3.08 倍と最も速くなった．これは dense モデルでは $\gamma + 1$ トークンの verify でも読む重みが変わらず，verify がほぼ無料（$c = 1.16$）だからである．
- 26B-A4B は AL が 3.46 と高いのに 1.35 倍（再計測で 1.41 倍）にとどまった．これは verify トークンの数だけ読む expert が増え，1 speculative step のコストが通常の 2.56 倍になるからである．帯域モデルの予測値 $c_{\text{pred}} = 2.78$ もこれと近い．外部の報告（1.49–1.60 倍）より低いのは，MTP 有効時に GPU が 52% の時間 idle で，CPU 側の処理が律速しているからと考えられる．
- 高負荷では，26B-A4B の MTP の利得が 1.25 倍から 1.51 倍へと逆に増えた．これは高負荷ではもともと expert の大半を読んでいるため，verify トークンを足しても読み出しバイトがほとんど増えないからである．
- n-gram は自由な文章で 0.95 倍とかえって遅くなったが，MTP はどのワークロードでも 3.0–3.5 倍だった．これは n-gram がプロンプト内のコピーにしか当たらないのに対し，学習済みの drafter は内容によらず次のトークンを予測できるからである．

■Q4：何が律速しているか． 小さいモデルはカーネルの固定 latency，長いコンテキストは（v0.32.0 では）FA4 の split-KV のバグ，高並列は prefill の計算と 1 ステップの処理効率が律速である．高並列での KV 容量の不足は TTFT を悪化させるが，スループットは決めていない．それぞれの対策を表 19 にまとめた．

付録 A　付録

参考文献

[1] Gemma Team, Google DeepMind. *Gemma 4 Technical Report*, 2026.

[2] Google. *Gemma 4 model card*. `https://ai.google.dev/gemma/docs/core/model_card_4`

[3] Hugging Face. *Welcome Gemma 4: Frontier multimodal intelligence on device*, 2026. `https://`

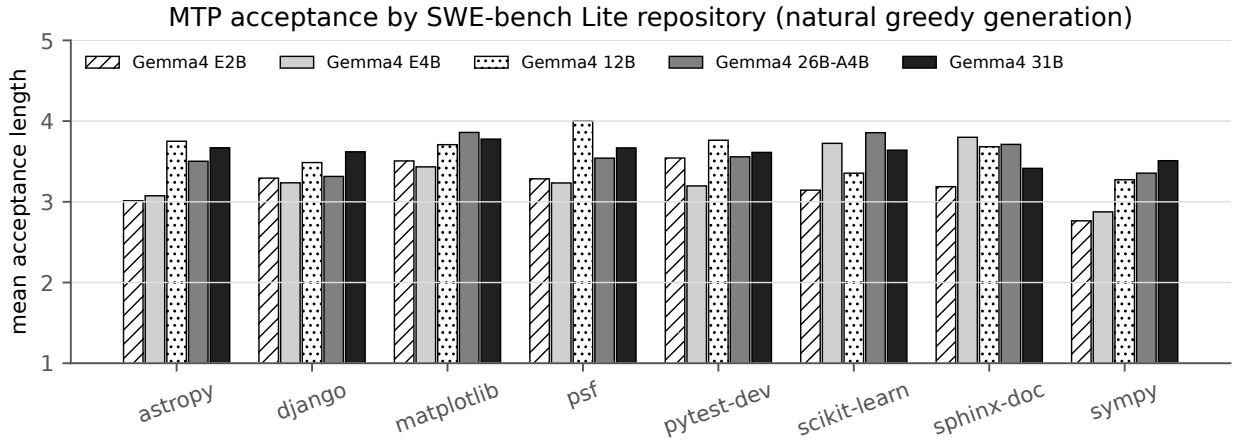


図 32: SWE-bench Lite のリポジトリ別の mean acceptance length（$\gamma = 4$，greedy 生成）.

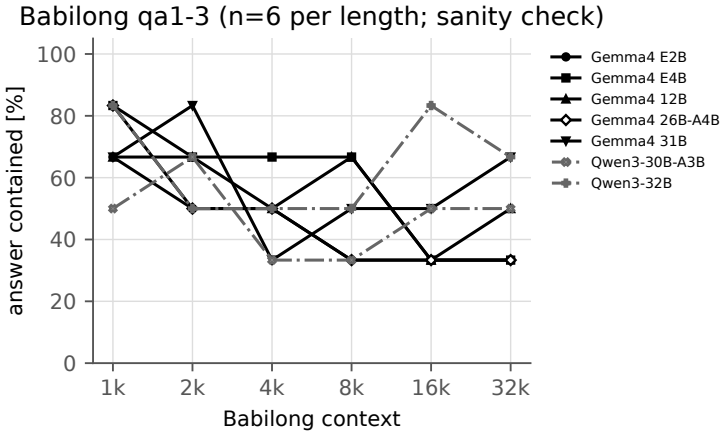


図 33: Babilong qa1–qa3 の正答率（出力冒頭 80 文字に正解語を含む割合，各長さ 6 件）．性能計測の健全性チェックであり，件数が少ないため精度の比較には使えない.

huggingface.co/blog/gemma4

[4] @googlegemma, X, 2026-05-05. https://x.com/googlegemma/status/2051694045869879749

[5] Google. *Multi-token prediction in Gemma 4*. https://blog.google/innovation-and-ai/technology/developers-tools/multi-token-prediction-gemma-4/

[6] vLLM PR #41745, *[Spec Decode] Add Gemma4 MTP speculative decoding support*. https://github.com/vllm-project/vllm/pull/41745

[7] kipply. *Transformer Inference Arithmetic*, 2022. https://kipp.ly/transformer-inference-arithmetic/

[8] Kotoba Technologies. *LLM Serving* を支える技術. https://zenn.dev/kotoba_tech/articles/98feb05f24c082

[9] Kuratov et al. *BABILong*. https://huggingface.co/datasets/RMT-team/babilong

[10] Jimenez et al. *SWE-bench*. https://huggingface.co/datasets/SWE-bench/SWE-bench_Lite

[11] NVIDIA. *H100 Tensor Core GPU Datasheet*.

[12] JarvisLabs. *Benchmarking Gemma 4 MTP vs DFlash on a Single H100*. https://jarvislabs.ai/blog/gemma-4-mtp-vs-dflash-benchmark

[13] InferenceBench. *Gemma 4 31B on H100: The Complete Inference Benchmark*. https://inferencebench.io/blog/gemma-4-31b-h100-complete-inference-benchmark/

[14] vLLM Issue #38887, *Gemma 4 E4B extremely slow on v0.19.0: forced TRITON_ATTN fallback.* https://github.com/vllm-project/vllm/issues/38887

[15] vLLM PR #38891, *[Gemma4] Allow per-layer attention backend selection for heterogeneous head dimensions.* https://github.com/vllm-project/vllm/pull/38891

[16] NVIDIA. *NVIDIA H100 Tensor Core GPU Architecture* (whitepaper), 2022.

[17] NVIDIA. *GPU Performance Background User's Guide.* https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html

[18] NVIDIA. *CUDA C++ Programming Guide.* https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[19] V. Volkov. *Understanding Latency Hiding on GPUs.* PhD thesis, UC Berkeley, Technical Report UCB/EECS-2016-143, 2016.

[20] A. Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways.* arXiv:2204.02311, 2022.